

Neural Ordinary Differential Equations

Ricky T. Q. Chen*, Yulia Rubanova*, Jesse Bettencourt*, David Duvenaud
University of Toronto, Vector Institute
{rtqichen, rubanova, jessebett, duvenaud}@cs.toronto.edu

Abstract

We introduce a new family of deep neural network models. Instead of specifying a discrete sequence of hidden layers, we parameterize the derivative of the hidden state using a neural network. The output of the network is computed using a black-box differential equation solver. These continuous-depth models have constant memory cost, adapt their evaluation strategy to each input, and can explicitly trade numerical precision for speed. We demonstrate these properties in continuous-depth residual networks and continuous-time latent variable models. We also construct continuous normalizing flows, a generative model that can train by maximum likelihood, without partitioning or ordering the data dimensions. For training, we show how to scalably backpropagate through any ODE solver, without access to its internal operations. This allows end-to-end training of ODEs within larger models.

1 Introduction

Models such as residual networks, recurrent neural network decoders, and normalizing flows build complicated transformations by composing a sequence of transformations to a hidden state:

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t) \quad (1)$$

where $t \in \{0 \dots T\}$ and $\mathbf{h}_t \in \mathbb{R}^D$. These iterative updates can be seen as an Euler discretization of a continuous transformation (Lu et al., 2017; Haber and Ruthotto, 2017; Ruthotto and Haber, 2018).

What happens as we add more layers and take smaller steps? In the limit, we parameterize the continuous dynamics of hidden units using an ordinary differential equation (ODE) specified by a neural network:

$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta) \quad (2)$$

Starting from the input layer $\mathbf{h}(0)$, we can define the output layer $\mathbf{h}(T)$ to be the solution to this ODE initial value problem at some time T . This value can be computed by a black-box differential equation solver, which evaluates the hidden unit dynamics f wherever necessary to determine the solution with the desired accuracy. Figure 1 contrasts these two approaches.

Defining and evaluating models using ODE solvers has several benefits:

Memory efficiency In Section 2, we show how to compute gradients of a scalar-valued loss with respect to all inputs of any ODE solver, *without backpropagating through the operations of the solver*. Not storing any intermediate quantities of the forward pass allows us to train our models with constant memory cost as a function of depth, a major bottleneck of training deep models.

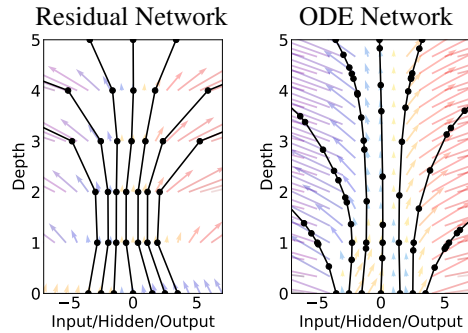


Figure 1: *Left:* A Residual network defines a discrete sequence of finite transformations. *Right:* A ODE network defines a vector field, which continuously transforms the state. *Both:* Circles represent evaluation locations.

神经常微分方程

Ricky T. Q. Chen*, Yulia Rubanova*, Jesse Bettencourt*, David Duvenaud 多伦多大学, Vector 研究所 {rtqichen, rubanova, jessebett, duvenaud}@cs.toronto.edu

摘要

我们引入了一类新的深度神经网络模型。我们不是指定一个离散的隐藏层序列, 而是使用神经网络对隐藏状态的导数进行参数化。网络的输出通过一个黑盒微分方程求解器计算。这些连续深度模型具有恒定的内存成本, 根据每个输入自适应调整评估策略, 并且可以明确地在数值精度和速度之间进行权衡。我们在连续深度残差网络和连续时间潜变量模型中展示了这些特性。我们还构建了连续归一化流, 这是一种生成模型, 可以通过最大似然方式训练, 而无需对数据维度进行分区或排序。在训练中, 我们展示了如何在不访问其内部操作的情况下, 对任何ODE求解器进行可扩展的反向传播。这允许在更大的模型中对ODE进行端到端训练。

1 引言

诸如残差网络、循环神经网络解码器和归一化流等模型通过将一系列变换组合到隐藏状态上来构建复杂的变换:

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t) \quad (1)$$

在 $t \in \{0 \dots T\}$ 和 $\mathbf{h}_t \in \mathbb{R}^D$ 。这些迭代更新可以被视为连续变换的欧拉离散化 (Lu 等, 2017; Haber 和 Ruthotto, 2017; Ruthotto 和 Haber, 2018)。

当我们添加更多层并采取更小的步长时, 会发生什么? 在极限情况下, 我们使用由神经网络指定的常微分方程 (ODE) 对隐藏单元的连续动态进行参数化:

$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta) \quad (2)$$

从输入层 $\mathbf{h}(0)$ 开始, 我们可以将输出层 $\mathbf{h}(T)$ 定义为在某个时间 T 的该常微分方程初值问题的解。这个值可以通过一个黑箱常微分方程求解器计算, 该求解器在必要时评估隐藏单元的动态 f 以确定所需精度的解。图 1 对比了这两种方法。

使用常微分方程求解器定义和评估模型有几个好处:

内存效率 在第2节中, 我们展示了如何计算标量值损失对任何ODE求解器的所有输入的梯度, *without backpropagating through the operations of the solver*。不存储前向传播的任何中间量使我们能够以与深度无关的恒定内存成本训练我们的模型, 这是训练深度模型的一个主要瓶颈。

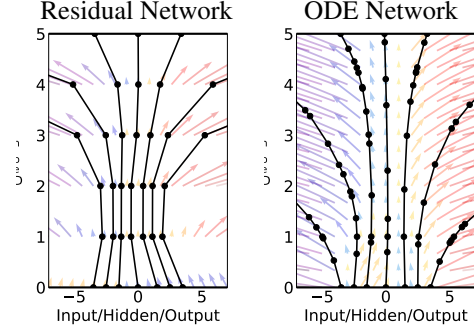


图 1: *Left*: 残差网络定义了一系列离散的有限变换。*Right*: ODE 网络定义了一个向量场, 它连续地变换状态。*Both*: 圆圈表示评估位置。

Adaptive computation Euler’s method is perhaps the simplest method for solving ODEs. There have since been more than 120 years of development of efficient and accurate ODE solvers (Runge, 1895; Kutta, 1901; Hairer et al., 1987). Modern ODE solvers provide guarantees about the growth of approximation error, monitor the level of error, and adapt their evaluation strategy on the fly to achieve the requested level of accuracy. This allows the cost of evaluating a model to scale with problem complexity. After training, accuracy can be reduced for real-time or low-power applications.

Scalable and invertible normalizing flows An unexpected side-benefit of continuous transformations is that the change of variables formula becomes easier to compute. In Section 4, we derive this result and use it to construct a new class of invertible density models that avoids the single-unit bottleneck of normalizing flows, and can be trained directly by maximum likelihood.

Continuous time-series models Unlike recurrent neural networks, which require discretizing observation and emission intervals, continuously-defined dynamics can naturally incorporate data which arrives at arbitrary times. In Section 5, we construct and demonstrate such a model.

2 Reverse-mode automatic differentiation of ODE solutions

The main technical difficulty in training continuous-depth networks is performing reverse-mode differentiation (also known as backpropagation) through the ODE solver. Differentiating through the operations of the forward pass is straightforward, but incurs a high memory cost and introduces additional numerical error.

We treat the ODE solver as a black box, and compute gradients using the *adjoint sensitivity method* (Pontryagin et al., 1962). This approach computes gradients by solving a second, augmented ODE backwards in time, and is applicable to all ODE solvers. This approach scales linearly with problem size, has low memory cost, and explicitly controls numerical error.

Consider optimizing a scalar-valued loss function $L()$, whose input is the result of an ODE solver:

$$L(\mathbf{z}(t_1)) = L\left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), t, \theta) dt\right) = L(\text{ODESolve}(\mathbf{z}(t_0), f, t_0, t_1, \theta)) \quad (3)$$

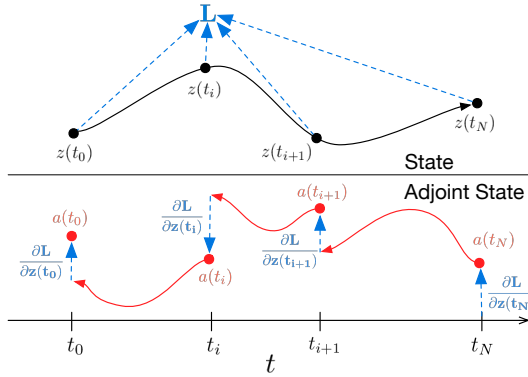


Figure 2: Reverse-mode differentiation of an ODE solution. The adjoint sensitivity method solves an augmented ODE backwards in time. The augmented system contains both the original state and the sensitivity of the loss with respect to the state. If the loss depends directly on the state at multiple observation times, the adjoint state must be updated in the direction of the partial derivative of the loss with respect to each observation.

To optimize L , we require gradients with respect to θ . The first step is to determining how the gradient of the loss depends on the hidden state $\mathbf{z}(t)$ at each instant. This quantity is called the *adjoint* $\mathbf{a}(t) = \partial L / \partial \mathbf{z}(t)$. Its dynamics are given by another ODE, which can be thought of as the instantaneous analog of the chain rule:

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}} \quad (4)$$

We can compute $\partial L / \partial \mathbf{z}(t_0)$ by another call to an ODE solver. This solver must run backwards, starting from the initial value of $\partial L / \partial \mathbf{z}(t_1)$. One complication is that solving this ODE requires the knowing value of $\mathbf{z}(t)$ along its entire trajectory. However, we can simply recompute $\mathbf{z}(t)$ backwards in time together with the adjoint, starting from its final value $\mathbf{z}(t_1)$.

Computing the gradients with respect to the parameters θ requires evaluating a third integral, which depends on both $\mathbf{z}(t)$ and $\mathbf{a}(t)$:

$$\frac{dL}{d\theta} = - \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt \quad (5)$$

The vector-Jacobian products $\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{z}}$ and $\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}$ in (4) and (5) can be efficiently evaluated by automatic differentiation, at a time cost similar to that of evaluating f . All integrals for solving $\mathbf{z}$, $\mathbf{a}$

自适应计算欧拉方法或许是求解常微分方程（ODEs）最简单的方法。从那时起，已经有超过120年的高效且精确的ODE求解器发展历史（Runge, 1895；Kutta, 1901；Hairer 等, 1987）。现代ODE求解器能够保证近似误差的增长，监控误差水平，并根据情况灵活调整计算策略以达到所需的精度水平。这使得模型的计算成本可以随问题的复杂性而变化。训练完成后，为实时或低功耗应用可以降低精度。

可扩展且可逆的归一化流连续变换的一个意外副作用是变量变化公式变得更容易计算。在第4节中，我们推导了这一结果，并利用它构建了一类新的可逆密度模型，该模型避免了一化流的单元瓶颈问题，并且可以直接通过最大似然进行训练。

连续时间序列模型与需要对观测和发射间隔进行离散化的循环神经网络不同，连续定义的动力学可以自然地整合在任意时间到达的数据。在第5节中，我们构建并演示了这样的模型。

2 ODE 解的反向自动微分

训练连续深度网络的主要技术难点是通过ODE求解器执行反向模式微分（也称为反向传播）。通过前向传递操作进行微分是直接的，但会产生高内存开销并引入额外的数值误差。

我们将ODE求解器视为一个黑箱，并使用*adjoint sensitivity method* (Pontryagin 等人, 1962) 计算梯度。这种方法通过向后求解第二个增强的ODE来计算梯度，并适用于所有ODE求解器。这种方法随问题规模线性扩展，内存开销低，并且可以显式控制数值误差。

考虑优化一个标量值的损失函数 $L()$ ，其输入是 ODE 求解器的结果：

$$L(\mathbf{z}(t_1)) = L\left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), t, \theta) dt\right) = L(\text{ODESolve}(\mathbf{z}(t_0), f, t_0, t_1, \theta)) \quad (3)$$

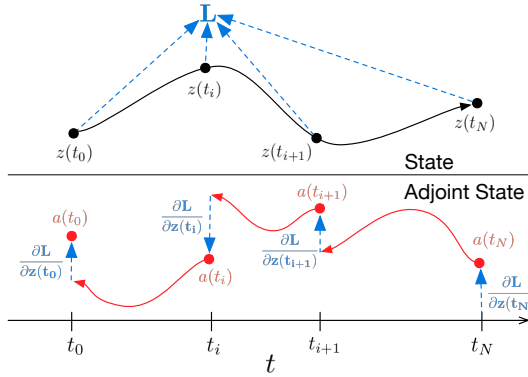


图 2：ODE 解的反向模式微分。伴随灵敏度方法通过时间向后求解一个扩展的 ODE。扩展系统包含原始状态以及损失对状态的灵敏度。如果损失在多个观测时间上直接依赖于状态，则必须沿着损失对每个观测的偏导方向更新伴随状态。

为了优化 L ，我们需要关于 θ 的梯度。第一步是确定损失的梯度如何依赖于每一时刻的隐藏状态 $\mathbf{z}(t)$ 。这个量被称为 *adjoint* $\mathbf{a}(t) = \partial L / \partial \mathbf{z}(t)$ 。它的动态由另一个常微分方程给出，可以被看作链式法则的瞬时对应物：

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}} \quad (4)$$

我们可以通过另一次调用 ODE 求解器来计算 $\partial L / \partial \mathbf{z}(t_0)$ 。这个求解器必须向后运行，从 $\partial L / \partial \mathbf{z}(t_1)$ 的初始值开始。一个复杂之处在于，求解这个 ODE 需要知道 $\mathbf{z}(t)$ 在整个轨迹上的数值。然而，我们可以简单地从其最终值 $\mathbf{z}(t_1)$ 开始，与伴随量一起向后重新计算 $\mathbf{z}(t)$ 。

计算关于参数 θ 的梯度需要评估第三个积分，该积分依赖于 $\mathbf{z}(t)$ 和 $\mathbf{a}(t)$ ：

$$\frac{dL}{d\theta} = - \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt \quad (5)$$

在(4)和(5)中的向量-雅可比 $\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{z}}$ 和 $\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}$ 可以通过自动微分高效地评估，其时间成本与评估 f 类似。求解 $\mathbf{z}$ 、 $\mathbf{a}$ 的所有积分

and $\frac{\partial L}{\partial \theta}$ can be computed in a single call to an ODE solver, which **concatenates** the original state, the adjoint, and the other partial derivatives into a single vector. Algorithm 1 shows how to construct the necessary dynamics, and call an ODE solver to compute all gradients at once.

Algorithm 1 Reverse-mode derivative of an ODE initial value problem

Input: dynamics parameters θ , start time t_0 , stop time t_1 , final state $\mathbf{z}(t_1)$, loss gradient $\partial L / \partial \mathbf{z}(t_1)$
 $s_0 = [\mathbf{z}(t_1), \frac{\partial L}{\partial \mathbf{z}(t_1)}, \mathbf{0}_{|\theta|}]$ ▷ Define initial augmented state
def aug_dynamics($[\mathbf{z}(t), \mathbf{a}(t), \cdot], t, \theta$): ▷ Define dynamics on augmented state
 return $[f(\mathbf{z}(t), t, \theta), -\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{z}}, -\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}]$ ▷ Compute vector-Jacobian products
 $[\mathbf{z}(t_0), \frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}] = \text{ODESolve}(s_0, \text{aug_dynamics}, t_1, t_0, \theta)$ ▷ Solve reverse-time ODE
return $\frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}$ ▷ Return gradients

Most ODE solvers have the option to output the state $\mathbf{z}(t)$ at multiple times. When the loss depends on these intermediate states, the reverse-mode derivative must be broken into a sequence of separate solves, one between each consecutive pair of output times (Figure 2). At each observation, the adjoint must be adjusted in the direction of the corresponding partial derivative $\partial L / \partial \mathbf{z}(t_i)$.

The results above extend those of [Stapor et al. \(2018, section 2.4.2\)](#). An extended version of Algorithm 1 including derivatives w.r.t. t_0 and t_1 can be found in Appendix C. Detailed derivations are provided in Appendix B. Appendix D provides Python code which computes all derivatives for `scipy.integrate.odeint` by extending the `autograd` automatic differentiation package. This code also supports all higher-order derivatives. We have since released a PyTorch ([Paszke et al., 2017](#)) implementation, including GPU-based implementations of several standard ODE solvers at github.com/rtqichen/torchdiffeq.

3 Replacing residual networks with ODEs for supervised learning

In this section, we experimentally investigate the training of neural ODEs for supervised learning.

Software To solve ODE initial value problems numerically, we use the implicit Adams method implemented in LSODE and VODE and interfaced through the `scipy.integrate` package. Being an implicit method, it has better guarantees than explicit methods such as Runge-Kutta but requires solving a nonlinear optimization problem at every step. This setup makes direct backpropagation through the integrator difficult. We implement the adjoint sensitivity method in Python’s `autograd` framework ([Maclaurin et al., 2015](#)). For the experiments in this section, we evaluated the hidden state dynamics and their derivatives on the GPU using Tensorflow, which were then called from the Fortran ODE solvers, which were called from Python `autograd` code.

Model Architectures We experiment with a small residual network which downsamples the input twice then applies 6 standard residual blocks [He et al. \(2016b\)](#), which are replaced by an `ODESolve` module in the ODE-Net variant. We also test a network with the same architecture but where gradients are backpropagated directly through a Runge-Kutta integrator, referred to as RK-Net. Table 1 shows test error, number of parameters, and memory cost. L denotes the number of layers in the ResNet, and $\tilde{L}$ is the number of function evaluations that the ODE solver requests in a single forward pass, which can be interpreted as an implicit number of layers. We find that ODE-Nets and RK-Nets can achieve around the same performance as the ResNet.

Table 1: Performance on MNIST. [†]From [LeCun et al. \(1998\)](#).

	Test Error	# Params	Memory	Time
1-Layer MLP [†]	1.60%	0.24 M	-	-
ResNet	0.41%	0.60 M	$\mathcal{O}(L)$	$\mathcal{O}(L)$
RK-Net	0.47%	0.22 M	$\mathcal{O}(\tilde{L})$	$\mathcal{O}(\tilde{L})$
ODE-Net	0.42%	0.22 M	$\mathcal{O}(1)$	$\mathcal{O}(\tilde{L})$

Error Control in ODE-Nets ODE solvers can approximately ensure that the output is within a given tolerance of the true solution. Changing this tolerance changes the behavior of the network. We first verify that error can indeed be controlled in Figure 3a. The time spent by the forward call is proportional to the number of function evaluations (Figure 3b), so tuning the tolerance gives us a

并且 $\frac{\partial L}{\partial \theta}$ 可以通过一次对 ODE 求解器的调用来计算，该求解器将原始状态、伴随变量和其他偏导数连接成一个向量。算法 1 展示了如何构建所需的动力学，并调用 ODE 求解器一次性计算所有梯度。

Algorithm 1 Reverse-mode derivative of an ODE initial value problem

Input: dynamics parameters θ , start time t_0 , stop time t_1 , final state $\mathbf{z}(t_1)$, loss gradient $\partial L / \partial \mathbf{z}(t_1)$
 $s_0 = [\mathbf{z}(t_1), \frac{\partial L}{\partial \mathbf{z}(t_1)}, \mathbf{0}_{|\theta|}]$ ▷ Define initial augmented state
def aug_dynamics($[\mathbf{z}(t), \mathbf{a}(t), \cdot], t, \theta$): ▷ Define dynamics on augmented state
 return $[f(\mathbf{z}(t), t, \theta), -\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{z}}, -\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}]$ ▷ Compute vector-Jacobian products
 $[\mathbf{z}(t_0), \frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}] = \text{ODESolve}(s_0, \text{aug_dynamics}, t_1, t_0, \theta)$ ▷ Solve reverse-time ODE
return $\frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}$ ▷ Return gradients

大多数 ODE 求解器都有选项可以在多个时间点输出状态 $\mathbf{z}(t)$ 。当损失依赖于这些中间状态时，反向模式导数必须被分解为一系列独立的求解，每次求解发生在每对连续输出时间之间（图 2）。在每次观测时，需要沿着对应的偏导数 $\partial L / \partial \mathbf{z}(t_i)$ 的方向调整伴随变量。

上述结果扩展了 Stapor 等人（2018，第 2.4.2 节）的研究。包含相对于 t_0 和 t_1 导数的扩展版本的算法 1 可以在附录 C 中找到。附录 B 提供了详细的推导过程。附录 D 提供了 Python 代码，通过扩展 autograd 自动微分包来计算 `scipy.integrate.odeint` 的所有导数。该代码还支持所有高阶导数。我们随后发布了一个 PyTorch（Paszke 等人，2017）的实现版本，包括多个标准 ODE 求解器的 GPU 实现，见 github.com/rtqichen/torchdiffeq。

3 用 ODE 替换残差网络进行监督学习

在本节中，我们通过实验研究神经常微分方程在监督学习中的训练。

软件 为了数值求解常微分方程初值问题，我们使用在 SODE 和 VODE 中实现的隐式 Adams 方法，并通过 `scipy.integrate` 包进行接口调用。作为一种隐式方法，它比显式方法（如 Runge-Kutta）具有更好的保证，但每一步都需要求解一个非线性优化问题。这个设置使得通过积分器进行直接反向传播变得困难。我们在 Python 的 autograd 框架中实现了伴随灵敏度方法（Maclaurin 等人，2015）。在本节的实验中，我们使用 Tensorflow 在 GPU 上评估了隐藏状态的动态及其导数，然后从 Fortran ODE 求解器中调用这些值，再由 Python autograd 代码调用。

模型架构 我们实验了一个小型残差网络，该网络将输入下采样两次，然后应用 6 个标准残差块 He 等 (2016b)，在 ODE-Net 变体中，这些残差块被 ODE 求解模块取代。我们还测试了一个具有相同架构的网络，但梯度直接通过 Runge-Kutta 积分器反向传播，

表 1: MNIST 上的性能。[†]来源: LeCun 等人 (1998)。

	Test Error	# Params	Memory	Time
1-Layer MLP [†]	1.60%	0.24 M	-	-
ResNet	0.41%	0.60 M	$\mathcal{O}(L)$	$\mathcal{O}(L)$
RK-Net	0.47%	0.22 M	$\mathcal{O}(\tilde{L})$	$\mathcal{O}(\tilde{L})$
ODE-Net	0.42%	0.22 M	$\mathcal{O}(1)$	$\mathcal{O}(\tilde{L})$

被称为 RK-Net。表 1 显示了测试误差、参数数量和内存开销。 L 表示 ResNet 中的层数， $\tilde{L}$ 是 ODE 求解器在一次前向传递中请求的函数评估次数，可以解释为隐含的层数。我们发现 ODE-Nets 和 RK-Nets 可以达到与 ResNet 大致相同的性能。

ODE-Net 中的误差控制 ODE 求解器可以近似确保输出在给定的真实解容差范围内。改变这个容差会改变网络的行为。我们首先在图 3a 中验证了误差确实可以被控制。前向调用所花费的时间与函数评估的次数成正比（图 3b），因此调整容差给了我们一个

trade-off between accuracy and computational cost. One could train with high accuracy, but switch to a lower accuracy at test time.

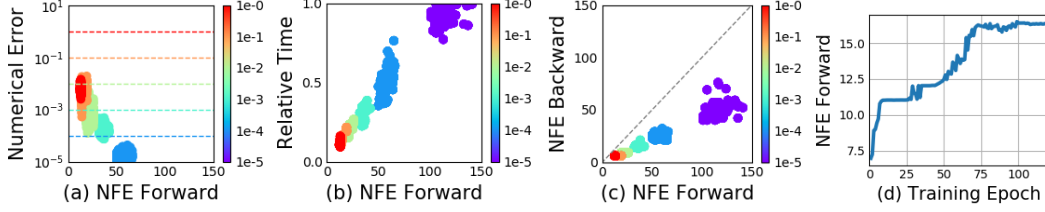


Figure 3: Statistics of a trained ODE-Net. (NFE = number of function evaluations.)

Figure 3c) shows a surprising result: the number of evaluations in the backward pass is roughly half of the forward pass. This suggests that the adjoint sensitivity method is not only more memory efficient, but also more computationally efficient than directly backpropagating through the integrator, because the latter approach will need to backprop through each function evaluation in the forward pass.

Network Depth It’s not clear how to define the ‘depth’ of an ODE solution. A related quantity is the number of evaluations of the hidden state dynamics required, a detail delegated to the ODE solver and dependent on the initial state or input. Figure 3d shows that the number of function evaluations increases throughout training, presumably adapting to increasing complexity of the model.

4 Continuous Normalizing Flows

The discretized equation (1) also appears in normalizing flows (Rezende and Mohamed, 2015) and the NICE framework (Dinh et al., 2014). These methods use the change of variables theorem to compute exact changes in probability if samples are transformed through a bijective function f :

$$\mathbf{z}_1 = f(\mathbf{z}_0) \implies \log p(\mathbf{z}_1) = \log p(\mathbf{z}_0) - \log \left| \det \frac{\partial f}{\partial \mathbf{z}_0} \right| \quad (6)$$

An example is the planar normalizing flow (Rezende and Mohamed, 2015):

$$\mathbf{z}(t+1) = \mathbf{z}(t) + u h(w^\top \mathbf{z}(t) + b), \quad \log p(\mathbf{z}(t+1)) = \log p(\mathbf{z}(t)) - \log \left| 1 + u^\top \frac{\partial h}{\partial \mathbf{z}} \right| \quad (7)$$

Generally, the main bottleneck to using the change of variables formula is computing of the determinant of the Jacobian $\partial f / \partial \mathbf{z}$, which has a cubic cost in either the dimension of $\mathbf{z}$, or the number of hidden units. Recent work explores the tradeoff between the expressiveness of normalizing flow layers and computational cost (Kingma et al., 2016; Tomczak and Welling, 2016; Berg et al., 2018).

Surprisingly, moving from a discrete set of layers to a continuous transformation simplifies the computation of the change in normalizing constant:

Theorem 1 (Instantaneous Change of Variables). *Let $\mathbf{z}(t)$ be a finite continuous random variable with probability $p(\mathbf{z}(t))$ dependent on time. Let $\frac{d\mathbf{z}}{dt} = f(\mathbf{z}(t), t)$ be a differential equation describing a continuous-in-time transformation of $\mathbf{z}(t)$. Assuming that f is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t , then the change in log probability also follows a differential equation,*

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr} \left(\frac{df}{d\mathbf{z}(t)} \right) \quad (8)$$

Proof in Appendix A. Instead of the log determinant in (6), we now only require a trace operation. Also unlike standard finite flows, the differential equation f does not need to be bijective, since if uniqueness is satisfied, then the entire transformation is automatically bijective.

As an example application of the instantaneous change of variables, we can examine the continuous analog of the planar flow, and its change in normalization constant:

$$\frac{d\mathbf{z}(t)}{dt} = u h(w^\top \mathbf{z}(t) + b), \quad \frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -u^\top \frac{\partial h}{\partial \mathbf{z}(t)} \quad (9)$$

精度与计算成本之间的权衡。可以在训练时使用高精度，但在测试时切换到较低精度。

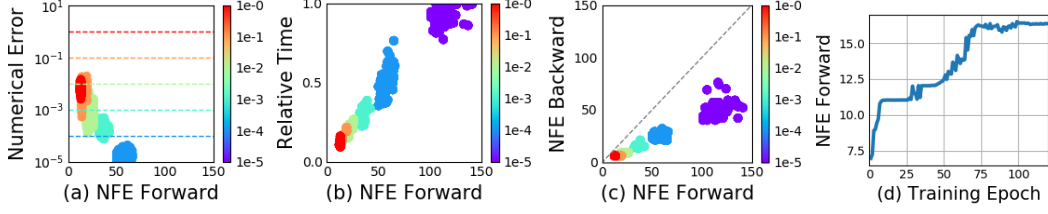


图 3: 训练后的 ODE-Net 的统计数据。(NFE = 函数评估次数。)

图3c) 显示了一个令人惊讶的结果：反向传递中的评估次数大约是前向传递的一半。这表明伴随灵敏度方法不仅在内存上更高效，而且在计算上也比直接通过积分器进行反向传播更高效，因为后者方法需要对前向传递中的每次函数评估进行反向传播。

网络深度 目前尚不清楚如何定义ODE解的“深度”。一个相关的量是所需的隐藏状态动力学评估次数，这一细节由ODE求解器决定，并依赖于初始状态或输入。图3d显示，函数评估的次数在整个训练过程中增加，可能是为了适应模型日益增加的复杂性。

4 连续归一化流

离散化的方程 (1) 也出现在归一化流 (Rezende 和 Mohamed, 2015) 以及 NICE 框架 (Dinh 等, 2014) 中。这些方法使用变量变换定理来计算如果样本通过双射函数 f 转换时概率的精确变化。

$$\mathbf{z}_1 = f(\mathbf{z}_0) \implies \log p(\mathbf{z}_1) = \log p(\mathbf{z}_0) - \log \left| \det \frac{\partial f}{\partial \mathbf{z}_0} \right| \quad (6)$$

一个例子是平面归一化流 (Rezende 和 Mohamed, 2015) :

$$\mathbf{z}(t+1) = \mathbf{z}(t) + uh(w^\top \mathbf{z}(t) + b), \quad \log p(\mathbf{z}(t+1)) = \log p(\mathbf{z}(t)) - \log \left| 1 + u^\top \frac{\partial h}{\partial \mathbf{z}} \right| \quad (7)$$

通常，使用变量变换公式的主要瓶颈是计算雅可比矩阵 $\partial f / \partial \mathbf{z}$ 的行列式，这在 $\mathbf{z}$ 的维度或隐藏单元数量上都有立方成本。最近的工作探讨了归一化流层的表达能力与计算成本之间的权衡 (Kingma 等, 2016; Tomczak 和 Welling, 2016; Berg 等, 2018)。

令人惊讶的是，从离散层集转向连续变换简化了归一化常数变化的计算：

定理 1 (变量的瞬时变化)。Let $\mathbf{z}(t)$ be a finite continuous random variable with probability $p(\mathbf{z}(t))$ dependent on time. Let $\frac{d\mathbf{z}}{dt} = f(\mathbf{z}(t), t)$ be a differential equation describing a continuous-in-time transformation of $\mathbf{z}(t)$. Assuming that f is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t , then the change in log probability also follows a differential equation,

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr} \left(\frac{df}{d\mathbf{z}(t)} \right) \quad (8)$$

证明见附录A。与公式(6)中的对数行列式不同，我们现在只需要一个迹操作。此外，与标准有限流不同，微分方程 f 不需要是双射，因为如果满足唯一性，则整个变换自动是双射的。

作为瞬时变量变化的示例应用，我们可以研究平面流的连续类比及其归一化常数的变化：

$$\frac{d\mathbf{z}(t)}{dt} = uh(w^\top \mathbf{z}(t) + b), \quad \frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -u^\top \frac{\partial h}{\partial \mathbf{z}(t)} \quad (9)$$

Given an initial distribution $p(\mathbf{z}(0))$, we can sample from $p(\mathbf{z}(t))$ and evaluate its density by solving this combined ODE.

Using multiple hidden units with linear cost While $\det$ is not a linear function, the trace function is, which implies $\text{tr}(\sum_n J_n) = \sum_n \text{tr}(J_n)$. Thus if our dynamics is given by a sum of functions then the differential equation for the log density is also a sum:

$$\frac{d\mathbf{z}(t)}{dt} = \sum_{n=1}^M f_n(\mathbf{z}(t)), \quad \frac{d \log p(\mathbf{z}(t))}{dt} = \sum_{n=1}^M \text{tr} \left(\frac{\partial f_n}{\partial \mathbf{z}} \right) \quad (10)$$

This means we can cheaply evaluate flow models having many hidden units, with a cost only linear in the number of hidden units M . Evaluating such ‘wide’ flow layers using standard normalizing flows costs $\mathcal{O}(M^3)$, meaning that standard NF architectures use many layers of only a single hidden unit.

Time-dependent dynamics We can specify the parameters of a flow as a function of t , making the differential equation $f(\mathbf{z}(t), t)$ change with t . This parameterization is a kind of hypernetwork (Ha et al., 2016). We also introduce a gating mechanism for each hidden unit, $\frac{d\mathbf{z}}{dt} = \sum_n \sigma_n(t) f_n(\mathbf{z})$ where $\sigma_n(t) \in (0, 1)$ is a neural network that learns when the dynamic $f_n(\mathbf{z})$ should be applied. We call these models continuous normalizing flows (CNF).

4.1 Experiments with Continuous Normalizing Flows

We first compare continuous and discrete planar flows at learning to sample from a known distribution. We show that a planar CNF with M hidden units can be at least as expressive as a planar NF with $K = M$ layers, and sometimes much more expressive.

Density matching We configure the CNF as described above, and train for 10,000 iterations using Adam (Kingma and Ba, 2014). In contrast, the NF is trained for 500,000 iterations using RMSprop (Hinton et al., 2012), as suggested by Rezende and Mohamed (2015). For this task, we minimize KL ($q(\mathbf{x}) \| p(\mathbf{x})$) as the loss function where q is the flow model and the target density $p(\cdot)$ can be evaluated. Figure 4 shows that CNF generally achieves lower loss.

Maximum Likelihood Training A useful property of continuous-time normalizing flows is that we can compute the reverse transformation for about the same cost as the forward pass, which cannot be said for normalizing flows. This lets us train the flow on a density estimation task by performing maximum likelihood estimation, which maximizes $\mathbb{E}_{p(\mathbf{x})} [\log q(\mathbf{x})]$ where $q(\cdot)$ is computed using the appropriate change of variables theorem, then afterwards reverse the CNF to generate random samples from $q(\mathbf{x})$.

For this task, we use 64 hidden units for CNF, and 64 stacked one-hidden-unit layers for NF. Figure 5 shows the learned dynamics. Instead of showing the initial Gaussian distribution, we display the

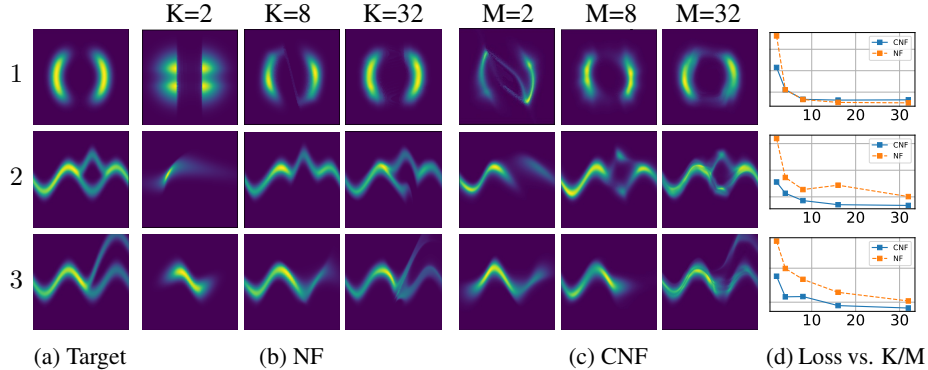


Figure 4: Comparison of normalizing flows versus continuous normalizing flows. The model capacity of normalizing flows is determined by their depth (K), while continuous normalizing flows can also increase capacity by increasing width (M), making them easier to train.

给定初始分布 $p(\mathbf{z}(0))$ ，我们可以从 $p(\mathbf{z}(t))$ 中采样并通过求解这个组合的常微分方程来评估其密度。

使用多个隐藏单元和线性代价 尽管行列式不是线性函数，但迹函数是线性函数，这意味着 $\text{tr}(\sum_n J_n) = \sum_n \text{tr}(J_n)$ 。因此，如果我们的动力学由函数的总和给出，那么对数密度的微分方程也是一个总和：

$$\frac{d\mathbf{z}(t)}{dt} = \sum_{n=1}^M f_n(\mathbf{z}(t)), \quad \frac{d \log p(\mathbf{z}(t))}{dt} = \sum_{n=1}^M \text{tr} \left(\frac{\partial f_n}{\partial \mathbf{z}} \right) \quad (10)$$

这意味着我们可以以低成本评估具有许多隐藏单元的流模型，成本仅与隐藏单元的数量呈线性关系 M 。使用标准归一化流评估这种“宽”流层的成本为 $\mathcal{O}(M^3)$ ，这意味着标准 NF 架构使用许多仅有一个隐藏单元的层。

时间依赖的动力学 我们可以将流的参数指定为 t 的函数，使微分方程 $f(\mathbf{z}(t), t)$ 随 t 变化。这种参数化是一种超网络 (Ha 等, 2016)。我们还为每个隐藏单元引入了门控机制， $\frac{d\mathbf{z}}{dt} = \sum_n \sigma_n(t) f_n(\mathbf{z})$ 其中 $\sigma_n(t) \in (0, 1)$ 是一个神经网络，用于学习何时应用动力学 $f_n(\mathbf{z})$ 。我们称这些模型为连续归一化流 (CNF)。

4.1 连续归一化流实验

我们首先比较了连续和平面流在从已知分布中学习采样的表现。我们展示了一个具有 M 个隐藏单元的平面 CNF 至少可以与一个拥有 $K = M$ 层的平面 NF 一样具有表现力，有时甚至更有表现力。

密度匹配 我们按照上述方法配置 CNF，并使用 Adam (Kingma 和 Ba, 2014) 训练 10,000 次迭代。相比之下，NF 按照 Rezende 和 Mohamed (2015) 的建议，使用 RMSprop (Hinton 等, 2012) 训练 500,000 次迭代。对于该任务，我们最小化 $\text{KL}(q(\mathbf{x}) \| p(\mathbf{x}))$ 作为损失函数，其中 q 是流模型，目标密度 $p(\cdot)$ 可以被评估。图 4 显示 CNF 通常能达到更低的损失。

最大似然训练 连续时间归一化流的一个有用特性是，我们可以以与前向传播大致相同的成本计算逆变换，而归一化流则无法做到这一点。这使我们能够通过执行最大似然估计在密度估计任务上训练流，从而最大化 $\mathbb{E}_{p(\mathbf{x})}[\log q(\mathbf{x})]$ ，其中 $q(\cdot)$ 是使用适当的变量变换定理计算的，然后之后逆转 CNF 从 $q(\mathbf{x})$ 中生成随机样本。

对于此任务，我们为 CNF 使用 64 个隐藏单元，为 NF 使用 64 个堆叠的一隐藏单元层。图 5 显示了学习到的动态。我们没有显示初始的高斯分布，而是展示了

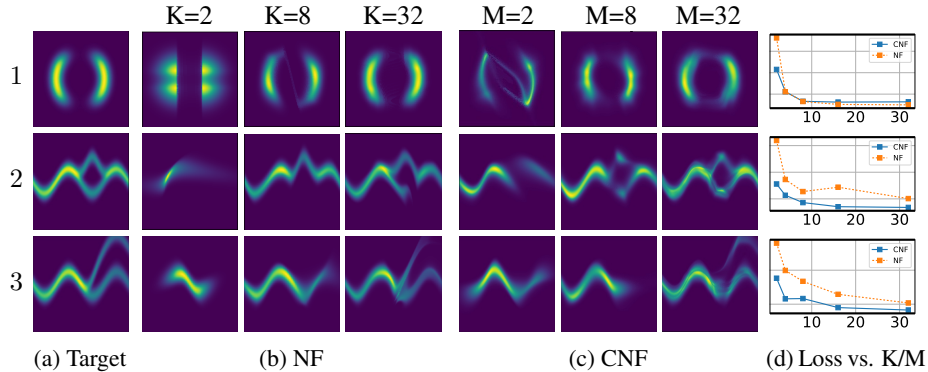


图4: 标准归一化流与连续归一化流的比较。标准归一化流的模型容量由其深度 (K) 决定，而连续归一化流也可以通过增加宽度 (M) 来提高容量，从而使其更易于训练。

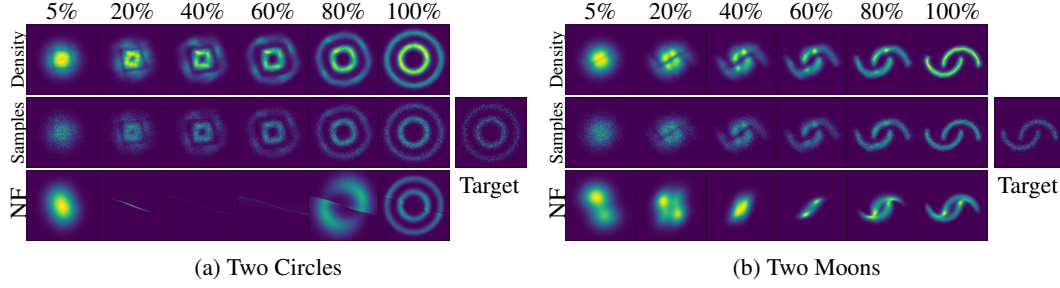


Figure 5: **Visualizing the transformation from noise to data.** Continuous-time normalizing flows are reversible, so we can train on a density estimation task and still be able to sample from the learned density efficiently.

transformed distribution after a small amount of time which shows the locations of the initial planar flows. Interestingly, to fit the Two Circles distribution, the CNF rotates the planar flows so that the particles can be evenly spread into circles. While the CNF transformations are smooth and interpretable, we find that NF transformations are very unintuitive and this model has difficulty fitting the two moons dataset in Figure 5b.

5 A generative latent function time-series model

Applying neural networks to irregularly-sampled data such as medical records, network traffic, or neural spiking data is difficult. Typically, observations are put into bins of fixed duration, and the latent dynamics are discretized in the same way. This leads to difficulties with missing data and ill-defined latent variables. Missing data can be addressed using generative time-series models (Álvarez and Lawrence, 2011; Futoma et al., 2017; Mei and Eisner, 2017; Soleimani et al., 2017a) or data imputation (Che et al., 2018). Another approach concatenates time-stamp information to the input of an RNN (Choi et al., 2016; Lipton et al., 2016; Du et al., 2016; Li, 2017).

We present a continuous-time, generative approach to modeling time series. Our model represents each time series by a latent trajectory. Each trajectory is determined from a local initial state, $\mathbf{z}_{t_0}$, and a global set of latent dynamics shared across all time series. Given observation times $t_0, t_1, \dots, t_N$ and an initial state $\mathbf{z}_{t_0}$, an ODE solver produces $\mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_N}$, which describe the latent state at each observation. We define this generative model formally through a sampling procedure:

$$\mathbf{z}_{t_0} \sim p(\mathbf{z}_{t_0}) \quad (11)$$

$$\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \dots, t_N) \quad (12)$$

$$\text{each } \mathbf{x}_{t_i} \sim p(\mathbf{x}|\mathbf{z}_{t_i}, \theta_x) \quad (13)$$

Function f is a time-invariant function that takes the value $\mathbf{z}$ at the current time step and outputs the gradient: $\partial \mathbf{z}(t)/\partial t = f(\mathbf{z}(t), \theta_f)$. We parametrize this function using a neural net. Because f is time-

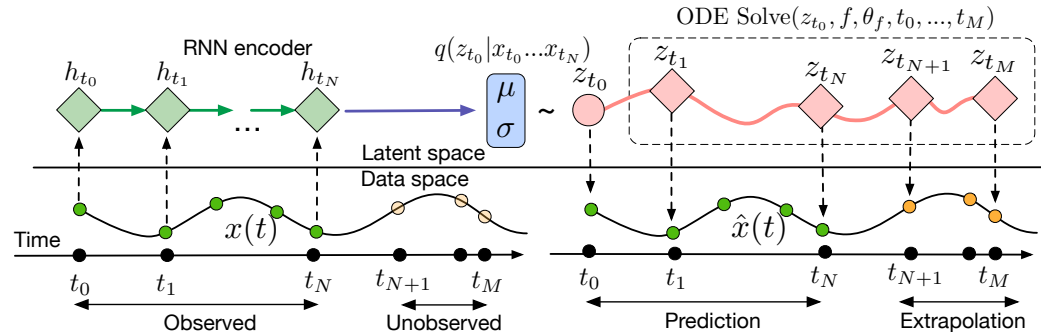


Figure 6: Computation graph of the latent ODE model.

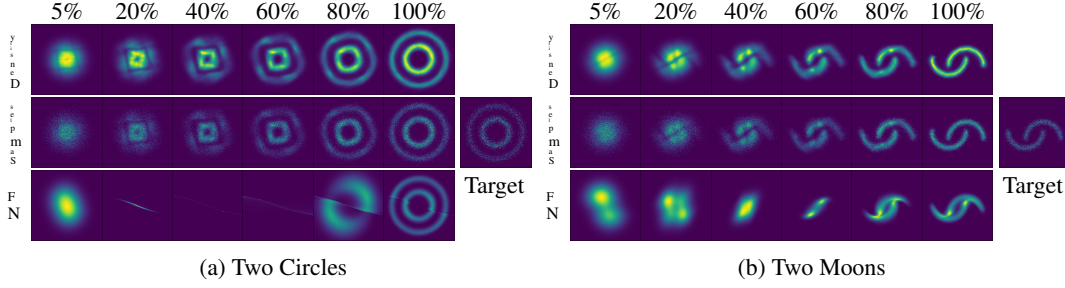


图5: 可视化从噪声到数据的转换。连续时间归一化流是可逆的, 因此我们可以在密度估计任务上进行训练, 并仍然能够有效地从学习到的密度中采样。

经过短时间的变换分布显示了初始平面流的位置。有趣的是, 为了拟合双圆分布, CNF 会旋转平面流, 使粒子能够均匀分布到圆中。虽然 CNF 的变换是平滑且可解释的, 但我们发现 NF 的变换非常不直观, 并且该模型在拟合图 5b 的双月数据集时存在困难。

5 一种生成型潜在功能时间序列模型

将神经网络应用于不规则采样的数据, 如病历、网络流量或神经脉冲数据, 是很困难的。通常, 将观测值放入固定时长的时间段中, 并以相同方式对潜在动态进行离散化。这会导致缺失数据和潜在变量定义不清的问题。可以使用生成时间序列模型 (Álvarez 和 Lawrence, 2011; Futoma 等, 2017; Mei 和 Eisner, 2017; Soleimani 等, 2017a) 或数据插补 (Che 等, 2018) 来处理缺失数据。另一种方法是时间戳信息与 RNN 的输入拼接 (Choi 等, 2016; Lipton 等, 2016; Du 等, 2016; Li, 2017)。

我们提出了一种用于建模时间序列的连续时间生成方法。我们的模型通过潜在轨迹表示每个时间序列。每条轨迹由局部初始状态 $\mathbf{z}_{t_0}$ 和一组在所有时间序列中共享的全局潜在动力学决定。给定观测时间 $t_0, t_1, \dots, t_N$ 和初始状态 $\mathbf{z}_{t_0}$, ODE 求解器生成 $\mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_N}$, 用于描述每个观测点的潜在状态。我们通过采样过程正式定义了这个生成模型:

$$\mathbf{z}_{t_0} \sim p(\mathbf{z}_{t_0}) \quad (11)$$

$$\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \dots, t_N) \quad (12)$$

$$\text{each } \mathbf{x}_{t_i} \sim p(\mathbf{x}|\mathbf{z}_{t_i}, \theta_x) \quad (13)$$

函数 f 是一个时间不变的函数, 在当前时间步的取值为 $\mathbf{z}$, 并输出梯度: $\partial \mathbf{z}(t)/\partial t = f(\mathbf{z}(t), \theta_f)$ 。我们使用神经网络对该函数进行参数化。因为 f 是时间-

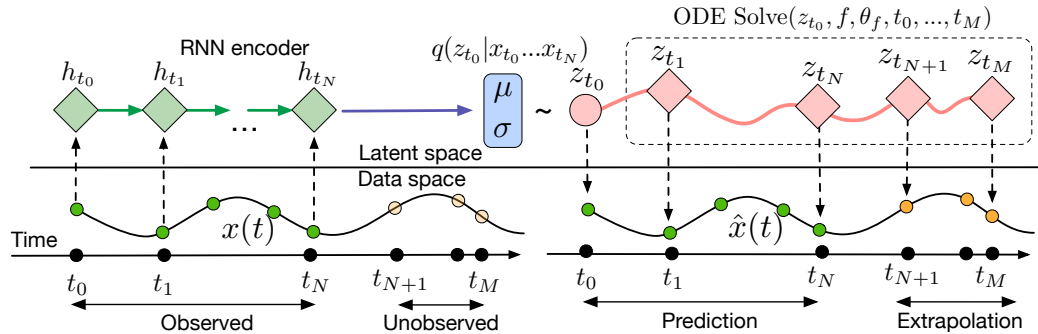


图6: 潜在ODE模型的计算图。

invariant, given any latent state $\mathbf{z}(t)$, the entire latent trajectory is uniquely defined. Extrapolating this latent trajectory lets us make predictions arbitrarily far forwards or backwards in time.

Training and Prediction We can train this latent-variable model as a variational autoencoder (Kingma and Welling, 2014; Rezende et al., 2014), with sequence-valued observations. Our recognition net is an RNN, which consumes the data sequentially backwards in time, and outputs $q_\phi(\mathbf{z}_0|\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$. A detailed algorithm can be found in Appendix E. Using ODEs as a generative model allows us to make predictions for arbitrary time points $t_1 \dots t_M$ on a continuous timeline.

Poisson Process likelihoods The fact that an observation occurred often tells us something about the latent state. For example, a patient may be more likely to take a medical test if they are sick. The rate of events can be parameterized by a function of the latent state: $p(\text{event at time } t | \mathbf{z}(t)) = \lambda(\mathbf{z}(t))$. Given this rate function, the likelihood of a set of independent observation times in the interval $[t_{\text{start}}, t_{\text{end}}]$ is given by an inhomogeneous Poisson process (Palm, 1943):

$$\log p(t_1 \dots t_N | t_{\text{start}}, t_{\text{end}}) = \sum_{i=1}^N \log \lambda(\mathbf{z}(t_i)) - \int_{t_{\text{start}}}^{t_{\text{end}}} \lambda(\mathbf{z}(t)) dt$$

We can parameterize $\lambda(\cdot)$ using another neural network. Conveniently, we can evaluate both the latent trajectory and the Poisson process likelihood together in a single call to an ODE solver. Figure 7 shows the event rate learned by such a model on a toy dataset.

A Poisson process likelihood on observation times can be combined with a data likelihood to jointly model all observations and the times at which they were made.

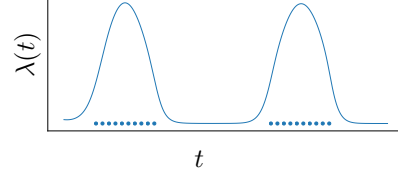


Figure 7: Fitting a latent ODE dynamics model with a Poisson process likelihood. Dots show event times. The line is the learned intensity $\lambda(t)$ of the Poisson process.

5.1 Time-series Latent ODE Experiments

We investigate the ability of the latent ODE model to fit and extrapolate time series. The recognition network is an RNN with 25 hidden units. We use a 4-dimensional latent space. We parameterize the dynamics function f with a one-hidden-layer network with 20 hidden units. The decoder computing $p(\mathbf{x}_{t_i} | \mathbf{z}_{t_i})$ is another neural network with one hidden layer with 20 hidden units. Our baseline was a recurrent neural net with 25 hidden units trained to minimize negative Gaussian log-likelihood. We trained a second version of this RNN whose inputs were concatenated with the time difference to the next observation to aid RNN with irregular observations.

Bi-directional spiral dataset We generated a dataset of 1000 2-dimensional spirals, each starting at a different point, sampled at 100 equally-spaced timesteps. The dataset contains two types of spirals: half are clockwise while the other half counter-clockwise. To make the task more realistic, we add gaussian noise to the observations.

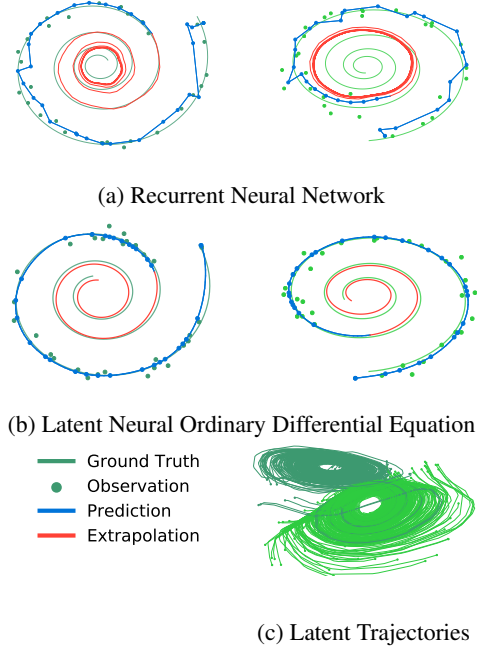


Figure 8: (a): Reconstruction and extrapolation of spirals with irregular time points by a recurrent neural network. (b): Reconstructions and extrapolations by a latent neural ODE. Blue curve shows model prediction. Red shows extrapolation. (c) A projection of inferred 4-dimensional latent ODE trajectories onto their first two dimensions. Color indicates the direction of the corresponding trajectory. The model has learned latent dynamics which distinguishes the two directions.

不变的，给定任何潜在状态 $\mathbf{z}(t)$ ，整个潜在轨迹是唯一确定的。外推这个潜在轨迹可以让我们在时间上任意向前或向后做出预测。

训练与预测 我们可以将这个潜变量模型作为变分自编码器（Kingma 和 Welling, 2014; Rezende 等, 2014）进行训练，处理序列值的观测数据。我们的识别网络是一个循环神经网络（RNN），它按时间顺序反向处理数据，并输出 $q_\phi(\mathbf{z}_0|\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$ 。详细算法可见附录 E。使用常微分方程（ODE）作为生成模型，使我们能够在连续时间轴上对任意时间点 $t_1 \dots t_M$ 进行预测。

泊松过程似然 观测事件的发生事实通常能告诉我们一些关于潜在状态的信息。例如，如果患者生病，他们可能更有可能进行医疗检查。事件的发生率可以由潜在状态的一个函数参数化： $p(\text{时间点的事件为 } t | \mathbf{z}(t)) = \lambda(\mathbf{z}(t))$ 。给定这个率函数，在区间 $[t_{\text{start}}, t_{\text{end}}]$ 内一组独立观测时间的似然由非齐次泊松过程给出（Palm, 1943）：

$$\log p(t_1 \dots t_N | t_{\text{start}}, t_{\text{end}}) = \sum_{i=1}^N \log \lambda(\mathbf{z}(t_i)) - \int_{t_{\text{start}}}^{t_{\text{end}}} \lambda(\mathbf{z}(t)) dt$$

我们可以使用另一个神经网络对 $\lambda(\cdot)$ 进行参数化。方便的是，我们可以评估潜在轨迹和

泊松过程似然在对 ODE 求解器的一次调用中一起计算。图7显示了该模型在一个玩具数据集上学到的事件率。

观察时间上的泊松过程似然可以与数据似然相结合，以联合建模所有观察结果及其发生的时间。

5.1 时间序列潜在 ODE 实验

我们研究了潜在 ODE 模型拟合和外推时间序列的能力。识别网络是一个具有 25 个隐藏单元的 RNN。我们使用一个四维的潜在空间。我们用一个具有 20 个隐藏单元的一隐藏层神经网络来参数化动力学函数 f 。计算 $p(\mathbf{x}_{t_i} | \mathbf{z}_{t_i})$ 的解码器是另一个具有 20 个隐藏单元的一隐藏层神经网络。我们的基线是一个具有 25 个隐藏单元的循环神经网络，训练目标是最小化高斯负对数似然。我们还训练了该 RNN 的第二个版本，其输入与到下一次观测的时间差拼接，以帮助 RNN 处理不规则观测。

双向螺旋数据集 我们生成了一个包含 1000 个二维螺旋的数据集，每个螺旋从不同的起点开始，在 100 个等间距时间步采样。数据集包含两种类型的螺旋：一半为顺时针，另一半为逆时针。为了使任务更现实，我们在观测中加入了高斯噪声。

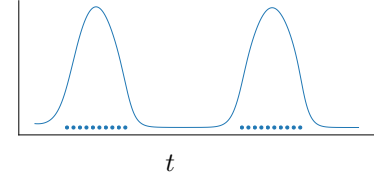


图7：使用泊松过程似然拟合潜在 ODE 动力学模型。点表示事件时间。线表示泊松过程的学习强度 $\lambda(t)$ 。

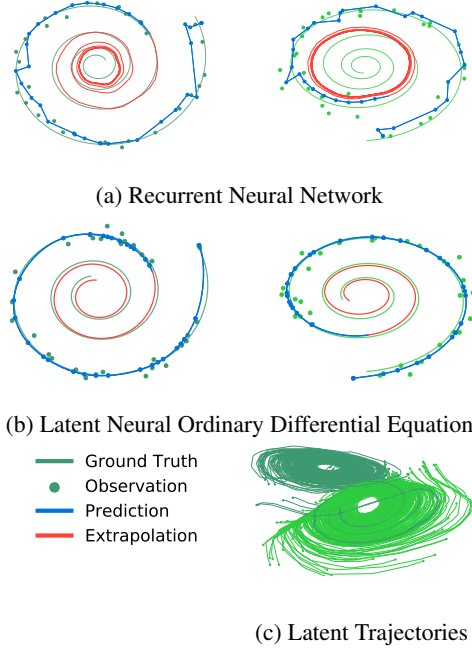


图8：(a)：通过循环神经网络对具有不规则时间点的螺旋进行重建和外推。(b)：通过潜在神经 ODE 进行的重建和外推。蓝色曲线表示模型预测，红色表示外推。(c) 推断的四维潜在 ODE 轨迹投影到其前两个维度的情况。颜色表示对应轨迹的方向。模型已经学习了区分两种方向的潜在动力学。



Figure 9: Data-space trajectories decoded from varying one dimension of $\mathbf{z}_{t_0}$. Color indicates progression through time, starting at purple and ending at red. Note that the trajectories on the left are counter-clockwise, while the trajectories on the right are clockwise.

Time series with irregular time points To generate irregular timestamps, we randomly sample points from each trajectory without replacement ($n = \{30, 50, 100\}$). We report predictive root-mean-squared error (RMSE) on 100 time points extending beyond those that were used for training. Table 2 shows that the latent ODE has substantially lower predictive RMSE.

Figure 8 shows examples of spiral reconstructions with 30 sub-sampled points. Reconstructions from the latent ODE were obtained by sampling from the posterior over latent trajectories and decoding it to data-space. Examples with varying number of time points are shown in Appendix F. We observed that reconstructions and extrapolations are consistent with the ground truth regardless of number of observed points and despite the noise.

Table 2: Predictive RMSE on test set

# Observations	30/100	50/100	100/100
RNN	0.3937	0.3202	0.1813
Latent ODE	0.1642	0.1502	0.1346

Latent space interpolation Figure 8c shows latent trajectories projected onto the first two dimensions of the latent space. The trajectories form two separate clusters of trajectories, one decoding to clockwise spirals, the other to counter-clockwise. Figure 9 shows that the latent trajectories change smoothly as a function of the initial point $\mathbf{z}(t_0)$, switching from a clockwise to a counter-clockwise spiral.

6 Scope and Limitations

Minibatching The use of mini-batches is less straightforward than for standard neural networks. One can still batch together evaluations through the ODE solver by concatenating the states of each batch element together, creating a combined ODE with dimension $D \times K$. In some cases, controlling error on all batch elements together might require evaluating the combined system K times more often than if each system was solved individually. However, in practice the number of evaluations did not increase substantially when using minibatches.

Uniqueness When do continuous dynamics have a unique solution? Picard’s existence theorem (Coddington and Levinson, 1955) states that the solution to an initial value problem exists and is unique if the differential equation is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t . This theorem holds for our model if the neural network has finite weights and uses Lipschitz nonlinearities, such as `tanh` or `relu`.

Setting tolerances Our framework allows the user to trade off speed for precision, but requires the user to choose an error tolerance on both the forward and reverse passes during training. For sequence modeling, the default value of $1.5\text{e-}8$ was used. In the classification and density estimation experiments, we were able to reduce the tolerance to $1\text{e-}3$ and $1\text{e-}5$, respectively, without degrading performance.

Reconstructing forward trajectories Reconstructing the state trajectory by running the dynamics backwards can introduce extra numerical error if the reconstructed trajectory diverges from the original. This problem can be addressed by checkpointing: storing intermediate values of $\mathbf{z}$ on the forward pass, and reconstructing the exact forward trajectory by re-integrating from those points. We did not find this to be a practical problem, and we informally checked that reversing many layers of continuous normalizing flows with default tolerances recovered the initial states.



图9: 从改变 $\mathbf{z}_{t_0}$ 的一个维度解码的数据空间轨迹。颜色表示时间的推进, 从紫色开始, 到红色结束。注意, 左侧的轨迹是逆时针方向, 而右侧的轨迹是顺时针方向。

具有不规则时间点的时间序列 为了生成不规则时间戳, 我们从每条轨迹中随机抽取点且不放回 ($n = \{30, 50, 100\}$)。我们在用于训练的时间点之外的100个时间点上报告预测均方根误差 (RMSE)。表2显示, 隐ODE的预测RMSE显著更低。

图8显示了具有30个子采样点的螺旋重建示例。潜在ODE的重建是通过从潜在轨迹的后验分布中采样并将其解码到数据空间获得的。具有不同时间点数量的示例显示在附录中。

表 2: 测试集上的预测均方根误差

# Observations	30/100	50/100	100/100
RNN	0.3937	0.3202	0.1813
Latent ODE	0.1642	0.1502	0.1346

附录 F。我们观察到, 无论观测点数量多少, 尽管存在噪声, 重建和外推都与真实情况一致。

潜在空间插值 图8c显示了投影到潜在空间前两维的潜在轨迹。这些轨迹形成了两个独立的轨迹簇, 一个解码为顺时针螺旋, 另一个解码为逆时针螺旋。图9显示潜在轨迹随着初始点 $\mathbf{z}(t_0)$ 的变化而平滑变化, 从顺时针螺旋切换到逆时针螺旋。

6 范围和局限性

小批量处理 小批量的使用比标准神经网络更不直接。仍然可以通过将每个批次元素的状态连接在一起, 从而通过ODE求解器将评估批量处理在一起, 创建一个维度为 $D \times K$ 的组合ODE。在某些情况下, 同时控制所有批次元素的误差可能需要对组合系统进行 K 倍于单独求解每个系统的评估。然而, 在实践中, 使用小批量时评估次数并没有显著增加。

唯一性 连续动力学什么时候具有唯一解? Picard 的存在性定理 (Coddington 和 Levinson, 1955) 指出, 如果微分方程在 $\mathbf{z}$ 上是统一 Lipschitz 连续的并且在 t 上是连续的, 则初值问题的解存在且唯一。如果神经网络具有有限权重并使用 Lipschitz 非线性函数 (例如 $\tanh$ 或 relu) , 该定理适用于我们的模型。

设置容差 我们的框架允许用户在速度和精度之间进行权衡, 但要求用户在训练过程中对正向和反向传递都选择一个误差容差。对于序列建模, 使用了默认值 $1.5\text{e-}8$ 。在分类和密度估计实验中, 我们能够将容差分别降低到 $1\text{e-}3$ 和 $1\text{e-}5$, 而不会降低性能。

重建前向轨迹 通过向后运行动力学来重建状态轨迹, 如果重建的轨迹偏离原始轨迹, 可能会引入额外的数值误差。这个问题可以通过检查点来解决: 在前向传递中存储 $\mathbf{z}$ 的中间值, 并从这些点重新积分以重建精确的前向轨迹。我们发现这并不是一个实际问题, 并且我们非正式地检查了, 在默认容差下逆转多层连续归一化流时, 可以恢复初始状态。

7 Related Work

The use of the adjoint method for training continuous-time neural networks was previously proposed (LeCun et al., 1988; Pearlmutter, 1995), though was not demonstrated practically. The interpretation of residual networks He et al. (2016a) as approximate ODE solvers spurred research into exploiting reversibility and approximate computation in ResNets (Chang et al., 2017; Lu et al., 2017). We demonstrate these same properties in more generality by directly using an ODE solver.

Adaptive computation One can adapt computation time by training secondary neural networks to choose the number of evaluations of recurrent or residual networks (Graves, 2016; Jernite et al., 2016; Figurnov et al., 2017; Chang et al., 2018). However, this introduces overhead both at training and test time, and extra parameters that need to be fit. In contrast, ODE solvers offer well-studied, computationally cheap, and generalizable rules for adapting the amount of computation.

Constant memory backprop through reversibility Recent work developed reversible versions of residual networks (Gomez et al., 2017; Haber and Ruthotto, 2017; Chang et al., 2017), which gives the same constant memory advantage as our approach. However, these methods require restricted architectures, which partition the hidden units. Our approach does not have these restrictions.

Learning differential equations Much recent work has proposed learning differential equations from data. One can train feed-forward or recurrent neural networks to approximate a differential equation (Raissi and Karniadakis, 2018; Raissi et al., 2018a; Long et al., 2017), with applications such as fluid simulation (Wiewel et al., 2018). There is also significant work on connecting Gaussian Processes (GPs) and ODE solvers (Schober et al., 2014). GPs have been adapted to fit differential equations (Raissi et al., 2018b) and can naturally model continuous-time effects and interventions (Soleimani et al., 2017b; Schulam and Saria, 2017). Ryder et al. (2018) use stochastic variational inference to recover the solution of a given stochastic differential equation.

Differentiating through ODE solvers The `dolfin` library (Farrell et al., 2013) implements adjoint computation for general ODE and PDE solutions, but only by backpropagating through the individual operations of the forward solver. The `Stan` library (Carpenter et al., 2015) implements gradient estimation through ODE solutions using forward sensitivity analysis. However, forward sensitivity analysis is quadratic-time in the number of variables, whereas the adjoint sensitivity analysis is linear (Carpenter et al., 2015; Zhang and Sandu, 2014). Melicher et al. (2017) used the adjoint method to train bespoke latent dynamic models.

In contrast, by providing a generic vector-Jacobian product, we allow an ODE solver to be trained end-to-end with any other differentiable model components. While use of vector-Jacobian products for solving the adjoint method has been explored in optimal control (Andersson, 2013; Andersson et al., In Press, 2018), we highlight the potential of a general integration of black-box ODE solvers into automatic differentiation (Baydin et al., 2018) for deep learning and generative modeling.

8 Conclusion

We investigated the use of black-box ODE solvers as a model component, developing new models for time-series modeling, supervised learning, and density estimation. These models are evaluated adaptively, and allow explicit control of the tradeoff between computation speed and accuracy. Finally, we derived an instantaneous version of the change of variables formula, and developed continuous-time normalizing flows, which can scale to large layer sizes.

9 Acknowledgements

We thank Wenyi Wang and Geoff Roeder for help with proofs, and Daniel Duckworth, Ethan Fetaya, Hossein Soleimani, Eldad Haber, Ken Caluwaerts, Daniel Flam-Shepherd, and Harry Braviner for feedback. We thank Chris Rackauckas, Dougal Maclaurin, and Matthew James Johnson for helpful discussions. We also thank Yuval Frommer for pointing out an unsupported claim about parameter efficiency.

7 相关工作

之前有人提出将伴随方法用于训练连续时间神经网络 (LeCun 等, 1988; Pearlmutter, 1995), 但尚未在实践中展示出来。He 等 (2016a) 将残差网络解释为近似的常微分方程求解器, 这促使了在 ResNet 中利用可逆性和近似计算的研究 (Chang 等, 2017; Lu 等, 2017)。我们通过直接使用常微分方程求解器, 展示了这些特性在更一般情况下的表现。

自适应计算 可以通过训练辅助神经网络来选择循环或残差网络的计算次数, 从而适应计算时间 (Graves, 2016; Jernite 等, 2016; Figurnov 等, 2017; Chang 等, 2018)。然而, 这会在训练和测试阶段引入额外的开销, 并且需要拟合额外的参数。相比之下, ODE 求解器提供了研究充分、计算成本低且可推广的规则, 用于调整计算量。

通过可逆性进行常量内存反向传播 最近的工作开发了可逆版本的残差网络 (Gomez 等, 2017; Haber 和 Ruthotto, 2017; Chang 等, 2017), 这提供了与我们的方法相同的常量内存优势。然而, 这些方法需要受限的架构, 这些架构会划分隐藏单元。我们的方法没有这些限制。

学习微分方程 最近的许多研究提出了从数据中学习微分方程的方法。可以训练前馈或递归神经网络来近似微分方程 (Raissi 和 Karniadakis, 2018; Raissi 等, 2018a; Long 等, 2017), 其应用包括流体模拟 (Wiewel 等, 2018)。在将高斯过程 (GPs) 与常微分方程求解器 (ODE solvers) 连接方面也有大量研究 (Schober 等, 2014)。高斯过程已被改造用于拟合微分方程 (Raissi 等, 2018b), 并且可以自然地建模连续时间效应和干预 (Soleimani 等, 2017b; Sc hulam 和 Saria, 2017)。Ryder 等 (2018) 使用随机变分推断来恢复给定随机微分方程的解。

通过 ODE 求解器进行微分 Dofin 库 (Farrell 等, 2013) 实现了一般 ODE 和 PDE 解的伴随计算, 但仅通过对前向求解器的各个操作进行反向传播来实现。Stan 库 (Carpenter 等, 2015) 通过前向敏感性分析实现了 ODE 解的梯度估计。然而, 前向敏感性分析的时间复杂度与变量数量平方成正比, 而伴随敏感性分析是线性的 (Carpenter 等, 2015; Zhang 和 Sandu, 2014)。Melicher 等 (2017) 使用伴随方法训练定制的潜在动态模型。

相比之下, 通过提供通用的向量-雅可比积, 我们允许 ODE 求解器与任何其他可微分模型组件进行端到端训练。虽然在最优控制中已经探索了利用向量-雅可比积来求解伴随方法 (Andersson, 2013; Andersson 等, 待出版, 2018), 我们强调了将黑箱 ODE 求解器通用整合到自动微分 (Baydin 等, 2018) 中以用于深度学习和生成模型的潜力。

8 结论

我们研究了将黑箱 ODE 求解器用作模型组件的情况, 开发了用于时间序列建模、监督学习和密度估计的新模型。这些模型可以自适应评估, 并允许明确控制计算速度与精度之间的权衡。最后, 我们推导了变量变化公式的瞬时版本, 并开发了连续时间归一化流, 可以扩展到较大的层规模。

9 致谢

我们感谢王文宜和 Geoff Roeder 在证明方面的帮助, 以及感谢 Daniel Duckworth、Ethan Fetaya、Hossein Soleimani、Eldad Haber、Ken Caluwaerts、Daniel Flam-Shepherd 和 Harry Braviner 的反馈。我们感谢 Chris Rackauckas、Dougal Maclaurin 和 Matthew James Johnson 的有益讨论。我们还感谢 Yuval Frommer 指出了关于参数效率的未经支持的声明。

References

- Mauricio A Álvarez and Neil D Lawrence. Computationally efficient convolved multiple output Gaussian processes. *Journal of Machine Learning Research*, 12(May):1459–1500, 2011.
- Brandon Amos and J Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning*, pages 136–145, 2017.
- Joel Andersson. *A general-purpose software framework for dynamic optimization*. PhD thesis, 2013.
- Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, In Press, 2018.
- Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18 (153):1–153, 2018.
- Rianne van den Berg, Leonard Hasenclever, Jakub M Tomczak, and Max Welling. Sylvester normalizing flows for variational inference. *arXiv preprint arXiv:1803.05649*, 2018.
- Bob Carpenter, Matthew D Hoffman, Marcus Brubaker, Daniel Lee, Peter Li, and Michael Betancourt. The Stan math library: Reverse-mode automatic differentiation in c++. *arXiv preprint arXiv:1509.07164*, 2015.
- Bo Chang, Lili Meng, Eldad Haber, Lars Ruthotto, David Begert, and Elliot Holtham. Reversible architectures for arbitrarily deep residual neural networks. *arXiv preprint arXiv:1709.03698*, 2017.
- Bo Chang, Lili Meng, Eldad Haber, Frederick Tung, and David Begert. Multi-level residual networks from dynamical systems view. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=SyJS-OgR->.
- Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. Recurrent neural networks for multivariate time series with missing values. *Scientific Reports*, 8(1):6085, 2018. URL <https://doi.org/10.1038/s41598-018-24271-9>.
- Edward Choi, Mohammad Taha Bahadori, Andy Schuetz, Walter F. Stewart, and Jimeng Sun. Doctor AI: Predicting clinical events via recurrent neural networks. In *Proceedings of the 1st Machine Learning for Healthcare Conference*, volume 56 of *Proceedings of Machine Learning Research*, pages 301–318. PMLR, 18–19 Aug 2016. URL <http://proceedings.mlr.press/v56/Choi16.html>.
- Earl A Coddington and Norman Levinson. *Theory of ordinary differential equations*. Tata McGraw-Hill Education, 1955.
- Laurent Dinh, David Krueger, and Yoshua Bengio. NICE: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- Nan Du, Hanjun Dai, Rakshit Trivedi, Utkarsh Upadhyay, Manuel Gomez-Rodriguez, and Le Song. Recurrent marked temporal point processes: Embedding event history to vector. In *International Conference on Knowledge Discovery and Data Mining*, pages 1555–1564. ACM, 2016.
- Patrick Farrell, David Ham, Simon Funke, and Marie Rognes. Automated derivation of the adjoint of high-level transient finite element programs. *SIAM Journal on Scientific Computing*, 2013.
- Michael Figurnov, Maxwell D Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry Vetrov, and Ruslan Salakhutdinov. Spatially adaptive computation time for residual networks. *arXiv preprint*, 2017.
- J. Futoma, S. Hariharan, and K. Heller. Learning to Detect Sepsis with a Multitask Gaussian Process RNN Classifier. *ArXiv e-prints*, 2017.
- Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. In *Advances in Neural Information Processing Systems*, pages 2211–2221, 2017.

参考文献

- Mauricio A Álvarez 和 Neil D Lawrence. 计算高效的卷积多输出高斯过程。 *Journal of Machine Learning Research*, 12(五月):1459–1500, 2011。
- Brandon Amos 和 J Zico Kolter. OptNet: 将可微分优化作为神经网络中的一层。发表于 *International Conference on Machine Learning*, 第136–145页, 2017年。
- Joel Andersson. *A general-purpose software framework for dynamic optimization*. 博士论文, 2013年。
- Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings 和 Moritz Diehl. CasADi——用于非线性优化和最优控制的软件框架。 *Mathematical Programming Computation*, 出版中, 2018。
- Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, 和 Jeffrey Mark Siskind. 机器学习中的自动微分: 综述。 *Journal of machine learning research*, 18 (153):1–153, 2018。
- Rianne van den Berg, Leonard Hasenclever, Jakub M Tomczak 和 Max Welling. 用于变分推断的 Sylvester 归一化流。 *arXiv preprint arXiv:1803.05649*, 2018。
- Bob Carpenter, Matthew D Hoffman, Marcus Brubaker, Daniel Lee, Peter Li 和 Michael Beta ncourt. 《Stan 数学库: C++ 中的反向模式自动微分》。 *arXiv preprint arXiv:1509.07164*, 2015。
- Bo Chang, Lili Meng, Eldad Haber, Lars Ruthotto, David Begert 和 Elliot Holtham. 可逆架构用于任意深度的残差神经网络。 *arXiv preprint arXiv:1709.03698*, 2017。
- Bo Chang, Lili Meng, Eldad Haber, Frederick Tung 和 David Begert. 从动力系统视角的多层残差网络. 发表在 *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=SyJS-OgR->. Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Song 和 Yan Liu. 用于具有缺失值的多变量时间序列的循环神经网络. *Scientific Reports*, 8(1):6085, 2018. URL <https://doi.org/10.1038/s41598-018-24271-9>.
- Edward Choi, Mohammad Taha Bahadori, Andy Schuetz, Walter F. Stewart 和 Jimeng Sun. 《Doctor AI: 通过循环神经网络预测临床事件》。发表于 *Proceedings of the 1st Machine Learning for Healthcare Conference*, *Proceedings of Machine Learning Research* 第 56 卷, 第 301–318 页. PMLR, 2016 年 8 月 18–19 日。网址: <http://proceedings.mlr.press/v56/Choi16.html>。
- 厄尔·A·科丁顿 和 诺曼·莱文森. *Theory of ordinary differential equations*. 塔塔麦格劳-希尔教育, 1955。
- Laurent Dinh, David Krueger 和 Yoshua Bengio. NICE: 非线性独立成分估计。 *arXiv preprint arXiv:1410.8516*, 2014。
- Nan Du, Hanjun Dai, Rakshit Trivedi, Utkarsh Upadhyay, Manuel Gomez-Rodriguez 和 Le Song. 循环标记时间点过程: 将事件历史嵌入向量。在 *International Conference on Knowledge Discovery and Data Mining*, 第 1555–1564 页. ACM, 2016。
- Patrick Farrell, David Ham, Simon Funke 和 Marie Rognes. 高阶瞬态有限元程序伴随的自动推导。 *SIAM Journal on Scientific Computing*, 2013。
- Michael Figurnov, Maxwell D Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry Vetrov 和 Ruslan Salakhutdinov. 残差网络的空间自适应计算时间。 *arXiv preprint*, 2017。
- J. Futoma, S. Hariharan 和 K. Heller. 使用多任务高斯过程 RNN 分类器学习检测败血症。 *ArXiv e-prints*, 2017。
- Aidan N Gomez, 孟烨任, Raquel Urtasun 和 Roger B Grosse. 可逆残差网络: 无需存储激活的反向传播。在 *Advances in Neural Information Processing Systems*, 第 2211–2221 页, 2017 年。

- Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- David Ha, Andrew Dai, and Quoc V Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.
- Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, 2017.
- E. Hairer, S.P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I – Nonstiff Problems*. Springer, 1987.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016a.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016b.
- Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent, 2012.
- Yacine Jernite, Edouard Grave, Armand Joulin, and Tomas Mikolov. Variable computation in recurrent neural networks. *arXiv preprint arXiv:1611.06188*, 2016.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. *International Conference on Learning Representations*, 2014.
- Diederik P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Advances in Neural Information Processing Systems*, pages 4743–4751, 2016.
- W. Kutta. Beitrag zur näherungsweise Integration totaler Differentialgleichungen. *Zeitschrift für Mathematik und Physik*, 46:435–453, 1901.
- Yann LeCun, D Touresky, G Hinton, and T Sejnowski. A theoretical framework for back-propagation. In *Proceedings of the 1988 connectionist models summer school*, volume 1, pages 21–28. CMU, Pittsburgh, Pa: Morgan Kaufmann, 1988.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Yang Li. Time-dependent representation for neural event sequence prediction. *arXiv preprint arXiv:1708.00065*, 2017.
- Zachary C Lipton, David Kale, and Randall Wetzell. Directly modeling missing data in sequences with RNNs: Improved classification of clinical time series. In *Proceedings of the 1st Machine Learning for Healthcare Conference*, volume 56 of *Proceedings of Machine Learning Research*, pages 253–270. PMLR, 18–19 Aug 2016. URL <http://proceedings.mlr.press/v56/Lipton16.html>.
- Z. Long, Y. Lu, X. Ma, and B. Dong. PDE-Net: Learning PDEs from Data. *ArXiv e-prints*, 2017.
- Yiping Lu, Aoxiao Zhong, Quanzheng Li, and Bin Dong. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. *arXiv preprint arXiv:1710.10121*, 2017.
- Dougal Maclaurin, David Duvenaud, and Ryan P Adams. Autograd: Reverse-mode differentiation of native Python. In *ICML workshop on Automatic Machine Learning*, 2015.
- Hongyuan Mei and Jason M Eisner. The neural Hawkes process: A neurally self-modulating multivariate point process. In *Advances in Neural Information Processing Systems*, pages 6757–6767, 2017.

亚历克斯·格雷夫斯。递归神经网络的自适应计算时间。 *arXiv preprint arXiv:1603.08983*, 2016。

David Ha, Andrew Dai 和 Quoc V Le。《超网络》。 *arXiv preprint arXiv:1609.09106*, 2016。

Eldad Haber 和 Lars Ruthotto. 深度神经网络的稳定架构. *Inverse Problems*, 34 (1):014004, 2017.

E. Hairer, S.P. Nørsett 和 G. Wanner. 《v4》。施普林格, 1987年。

何凯明, 张翔宇, 任少卿, 孙剑. 用于图像识别的深度残差学习. 收录于 *Proceedings of the IEEE conference on computer vision and pattern recognition*, 第770–778页, 2016a.

何凯明, 张翔宇, 任少卿, 孙剑。《深度残差网络中的身份映射》。收录于 *European conference on computer vision*, 第630–645页。施普林格, 2016b。

Geoffrey Hinton, Nitish Srivastava 和 Kevin Swersky。《机器学习的神经网络》第6a讲：小批量梯度下降概述, 2012年。

Yacine Jernite, Edouard Grave, Armand Joulin 和 Tomas Mikolov。循环神经网络中的可变计算。 *arXiv preprint arXiv:1611.06188*, 2016。

Diederik P Kingma 和 Jimmy Ba。Adam：一种随机优化方法。 *arXiv preprint arXiv:1412.6980*, 2014。

Diederik P. Kingma 和 Max Welling。自编码变分贝叶斯。 *International Conference on Learning Representations*, 2014。

Diederik P Kingma、Tim Salimans、Rafal Jozefowicz、Xi Chen、Ilya Sutskever 和 Max Welling。使用逆自回归流改进的变分推断。在 *Advances in Neural Information Processing Systems*, 第 4743–4751 页, 2016 年。

W. Kutta. 对全微分方程近似积分的贡献。 *Zeitschrift für Mathematik und Physik*, 46:435–453, 1901.

Yann LeCun, D Touresky, G Hinton 和 T Sejnowski. 反向传播的理论框架。收录于 *Proceedings of the 1988 connectionist models summer school*, 第1卷, 第21–28页。匹兹堡, 宾夕法尼亚州: Morgan Kaufmann, 1988年。

Yann LeCun, Léon Bottou, Yoshua Bengio 和 Patrick Haffner。基于梯度的学习在文档识别中的应用。 *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Yang Li. 用于神经事件序列预测的时间依赖表示. *arXiv preprint arXiv:1708.00065*, 2017.

Zachary C Lipton、David Kale 和 Randall Wetzal。使用 RNN 直接建模序列中的缺失数据：提高临床时间序列的分类。在 *Proceedings of the 1st Machine Learning for Healthcare Conference* 中, 第 56 卷 *Proceedings of Machine Learning Research*, 第 253–270 页。PMLR, 2016 年 8 月 18–19 日。网址 <http://proceedings.mlr.press/v56/Lipton16.html>。

Z. Long, Y. Lu, X. Ma, 和 B. Dong. PDE-Net：从数据中学习偏微分方程。 *ArXiv e-prints*, 2017.

卢一平, 钟奥晓, 李全正, 董斌. 超越有限层神经网络：连接深度架构与数值微分方程. *arXiv preprint arXiv:1710.10121*, 2017.

Dougal Maclaurin、David Duvenaud 和 Ryan P Adams。《Autograd：Python 原生的反向模式微分》，发表于 *ICML workshop on Automatic Machine Learning*, 2015 年。

梅鸿元 和 Jason M Eisner。神经霍克斯过程：一种神经自我调节的多变量点过程。《v26》，第6757–6767页, 2017年。

- Valdemar Melicher, Tom Haber, and Wim Vanroose. Fast derivatives of likelihood functionals for ODE based models using adjoint-state method. *Computational Statistics*, 32(4):1621–1643, 2017.
- Conny Palm. Intensitätsschwankungen im fernsprecher. *Ericsson Technics*, 1943.
- Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- Barak A Pearlmutter. Gradient calculations for dynamic recurrent neural networks: A survey. *IEEE Transactions on Neural networks*, 6(5):1212–1228, 1995.
- Lev Semenovich Pontryagin, EF Mishchenko, VG Boltyanskii, and RV Gamkrelidze. The mathematical theory of optimal processes. 1962.
- M. Raissi and G. E. Karniadakis. Hidden physics models: Machine learning of nonlinear partial differential equations. *Journal of Computational Physics*, pages 125–141, 2018.
- Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Multistep neural networks for data-driven discovery of nonlinear dynamical systems. *arXiv preprint arXiv:1801.01236*, 2018a.
- Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Numerical Gaussian processes for time-dependent and nonlinear partial differential equations. *SIAM Journal on Scientific Computing*, 40(1):A172–A198, 2018b.
- Danilo J Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1278–1286, 2014.
- Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. *arXiv preprint arXiv:1505.05770*, 2015.
- C. Runge. Über die numerische Auflösung von Differentialgleichungen. *Mathematische Annalen*, 46: 167–178, 1895.
- Lars Ruthotto and Eldad Haber. Deep neural networks motivated by partial differential equations. *arXiv preprint arXiv:1804.04272*, 2018.
- T. Ryder, A. Golightly, A. S. McGough, and D. Prangle. Black-box Variational Inference for Stochastic Differential Equations. *ArXiv e-prints*, 2018.
- Michael Schober, David Duvenaud, and Philipp Hennig. Probabilistic ODE solvers with Runge-Kutta means. In *Advances in Neural Information Processing Systems 25*, 2014.
- Peter Schulam and Suchi Saria. What-if reasoning with counterfactual Gaussian processes. *arXiv preprint arXiv:1703.10651*, 2017.
- Hossein Soleimani, James Hensman, and Suchi Saria. Scalable joint models for reliable uncertainty-aware event prediction. *IEEE transactions on pattern analysis and machine intelligence*, 2017a.
- Hossein Soleimani, Adarsh Subbaswamy, and Suchi Saria. Treatment-response models for counterfactual reasoning with continuous-time, continuous-valued interventions. *arXiv preprint arXiv:1704.02038*, 2017b.
- Jos Stam. Stable fluids. In *Proceedings of the 26th annual conference on Computer graphics and interactive techniques*, pages 121–128. ACM Press/Addison-Wesley Publishing Co., 1999.
- Paul Stapor, Fabian Froehlich, and Jan Hasenauer. Optimization and uncertainty analysis of ODE models using second order adjoint sensitivity analysis. *bioRxiv*, page 272005, 2018.
- Jakub M Tomczak and Max Welling. Improving variational auto-encoders using Householder flow. *arXiv preprint arXiv:1611.09630*, 2016.
- Steffen Wiewel, Moritz Becher, and Nils Thuerey. Latent-space physics: Towards learning the temporal evolution of fluid flow. *arXiv preprint arXiv:1802.10123*, 2018.
- Hong Zhang and Adrian Sandu. Fatode: a library for forward, adjoint, and tangent linear integration of ODEs. *SIAM Journal on Scientific Computing*, 36(5):C504–C523, 2014.

Valdemar Melicher、Tom Haber 和 Wim Vanroose。基于 ODE 模型的似然泛函快速导数计算：使用伴随状态方法。 *Computational Statistics*, 32(4):1621–1643, 2017。

康尼·帕尔姆。《电话通信中的强度波动》， *Ericsson Technics*, 1943年。

Adam Paszke、Sam Gross、Soumith Chintala、Gregory Chanan、Edward Yang、Zachary DeVito、Zeming Lin、Alban Desmaison、Luca Antiga 和 Adam Lerer。PyTorch 中的自动微分。2017 年。

巴拉克·A 珀尔马特。动态递归神经网络的梯度计算综述。 *IEEE Transactions on Neural networks*, 6(5):1212–1228, 1995。

列夫·谢苗诺维奇·庞特里亚金、EF·米先科、VG·博尔廷斯基，以及RV·甘克雷利泽。最优过程的数学理论。1962年。

M. Raissi 和 G. E. Karniadakis。隐藏物理模型：非线性偏微分方程的机器学习。 *Journal of Computational Physics*, 第125–141页，2018年。

Maziar Raissi、Paris Perdikaris 和 George Em Karniadakis。用于数据驱动非线性动力系统发现的多步神经网络。 *arXiv preprint arXiv:1801.01236*, 2018a。

Maziar Raissi、Paris Perdikaris 和 George Em Karniadakis。用于时变和非线性偏微分方程的数值高斯过程。 *SIAM Journal on Scientific Computing*, 40(1):A172–A198, 2018b。

Danilo J Rezende, Shakir Mohamed, 和 Daan Wierstra。深度生成模型中的随机反向传播和近似推断。在 *Proceedings of the 31st International Conference on Machine Learning*, 第1278–1286页，2014 年。

Danilo Jimenez Rezende 和 Shakir Mohamed。使用归一化流的变分推断。 *arXiv preprint arXiv:1505.05770*, 2015。

C. Runge。关于微分方程的数值求解。 *Mathematische Annalen*, 46: 167–178, 1895。

Lars Ruthotto 和 Eldad Haber。受偏微分方程启发的深度神经网络。 *arXiv preprint arXiv:1804.04272*, 2018。

T. Ryder, A. Golightly, A. S. McGough 和 D. Prangle。面向随机微分方程的黑箱变分推断。 *ArXiv e-prints*, 2018。

Michael Schober、David Duvenaud 和 Philipp Hennig。《带有 Runge-Kutta 均值的概率 ODE 求解器》，发表于 *Advances in Neural Information Processing Systems* 25, 2014 年。

Peter Schulam 和 Suchi Saria。使用反事实高斯过程进行假设推理。 *arXiv preprint arXiv:1703.10651*, 2017。

Hossein Soleimani、James Hensman 和 Suchi Saria。用于可靠的不确定性感知事件预测的可扩展联合模型。 *IEEE transactions on pattern analysis and machine intelligence*, 2017a。

Hossein Soleimani, Adarsh Subbaswamy 和 Suchi Saria。用于连续时间、连续值干预的反事实推理的治疗响应模型。 *arXiv preprint arXiv:1704.02038*, 2017b。

Jos Stam。《稳定流体》。载于 *Proceedings of the 26th annual conference on Computer graphics and interactive techniques*, 第121–128页。ACM出版社/Addison-Wesley出版社，1999年。
Paul Stapor、Fabian Froehlich 和 Jan Hasenauer。使用二阶伴随敏感性分析对常微分方程模型进行优化和不确定性分析。 *bioRxiv*, 第 272005 页，2018 年。

Jakub M Tomczak 和 Max Welling。使用 Householder 流改进变分自编码器。 *arXiv preprint arXiv:1611.09630*, 2016。

Steffen Wiewel、Moritz Becher 和 Nils Thuerey。《潜在空间物理学：迈向学习流体流动的时间演变》， *arXiv preprint arXiv:1802.10123*, 2018 年。

张洪 和 Adrian Sandu。Fatode: 一个用于常微分方程的前向、伴随和切线线性积分的库。 *SIAM Journal on Scientific Computing*, 36(5):C504–C523, 2014。

Appendix A Proof of the Instantaneous Change of Variables Theorem

Theorem (Instantaneous Change of Variables). *Let $\mathbf{z}(t)$ be a finite continuous random variable with probability $p(\mathbf{z}(t))$ dependent on time. Let $\frac{d\mathbf{z}}{dt} = f(\mathbf{z}(t), t)$ be a differential equation describing a continuous-in-time transformation of $\mathbf{z}(t)$. Assuming that f is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t , then the change in log probability also follows a differential equation:*

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr} \left(\frac{df}{d\mathbf{z}}(t) \right)$$

Proof. To prove this theorem, we take the infinitesimal limit of finite changes of $\log p(\mathbf{z}(t))$ through time. First we denote the transformation of $\mathbf{z}$ over an ε change in time as

$$\mathbf{z}(t + \varepsilon) = T_\varepsilon(\mathbf{z}(t)) \quad (14)$$

We assume that f is Lipschitz continuous in $\mathbf{z}(t)$ and continuous in t , so every initial value problem has a unique solution by Picard's existence theorem. We also assume $\mathbf{z}(t)$ is bounded. These conditions imply that f , T_ε , and $\frac{\partial}{\partial \mathbf{z}} T_\varepsilon$ are all bounded. In the following, we use these conditions to exchange limits and products.

We can write the differential equation $\frac{\partial \log p(\mathbf{z}(t))}{\partial t}$ using the discrete change of variables formula, and the definition of the derivative:

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = \lim_{\varepsilon \rightarrow 0^+} \frac{\log p(\mathbf{z}(t)) - \log |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))| - \log p(\mathbf{z}(t))}{\varepsilon} \quad (15)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\log |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|}{\varepsilon} \quad (16)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\frac{\partial}{\partial \varepsilon} \log |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|}{\frac{\partial}{\partial \varepsilon} \varepsilon} \quad (\text{by L'Hôpital's rule}) \quad (17)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\frac{\partial}{\partial \varepsilon} |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|}{|\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|} \quad \left(\left. \frac{\partial \log(\mathbf{z})}{\partial \mathbf{z}} \right|_{\mathbf{z}=\mathbf{1}} = 1 \right) \quad (18)$$

$$= - \underbrace{\left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \left| \det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right| \right)}_{\text{bounded}} \underbrace{\left(\lim_{\varepsilon \rightarrow 0^+} \frac{1}{|\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|} \right)}_{=1} \quad (19)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \left| \det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right| \quad (20)$$

The derivative of the determinant can be expressed using Jacobi's formula, which gives

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = - \lim_{\varepsilon \rightarrow 0^+} \text{tr} \left(\text{adj} \left(\frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \quad (21)$$

$$= - \text{tr} \left(\underbrace{\left(\lim_{\varepsilon \rightarrow 0^+} \text{adj} \left(\frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \right)}_{=I} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \right) \quad (22)$$

$$= - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \quad (23)$$

Substituting T_ε with its Taylor series expansion and taking the limit, we complete the proof.

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} (\mathbf{z} + \varepsilon f(\mathbf{z}(t), t) + \mathcal{O}(\varepsilon^2) + \mathcal{O}(\varepsilon^3) + \dots) \right) \quad (24)$$

$$= - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \left(I + \frac{\partial}{\partial \mathbf{z}} \varepsilon f(\mathbf{z}(t), t) + \mathcal{O}(\varepsilon^2) + \mathcal{O}(\varepsilon^3) + \dots \right) \right) \quad (25)$$

$$= - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \left(\frac{\partial}{\partial \mathbf{z}} f(\mathbf{z}(t), t) + \mathcal{O}(\varepsilon) + \mathcal{O}(\varepsilon^2) + \dots \right) \right) \quad (26)$$

$$= - \text{tr} \left(\frac{\partial}{\partial \mathbf{z}} f(\mathbf{z}(t), t) \right) \quad (27)$$

□

附录 A 瞬时变量变换定理的证明

定理（变量的瞬时变化）。Let $\mathbf{z}(t)$ be a finite continuous random variable with probability $p(\mathbf{z}(t))$ dependent on time. Let $\frac{d\mathbf{z}}{dt} = f(\mathbf{z}(t), t)$ be a differential equation describing a continuous-in-time transformation of $\mathbf{z}(t)$. Assuming that f is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t , then the change in log probability also follows a differential equation:

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr} \left(\frac{df}{d\mathbf{z}}(t) \right)$$

Proof. 为了证明这个定理，我们取随时间对数 $p(\mathbf{z}(t))$ 的有限变化的无穷小极限。首先我们表示 $\mathbf{z}$ 在时间 ε 变化下的变换为

$$\mathbf{z}(t + \varepsilon) = T_\varepsilon(\mathbf{z}(t)) \quad (14)$$

我们假设 f 在 $\mathbf{z}(t)$ 上是 Lipschitz 连续的，在 t 上是连续的，因此根据 Picard 的存在定理，每个初值问题都有唯一解。我们还假设 $\mathbf{z}(t)$ 是有界的。这些条件意味着 f 、 T_ε 和 $\frac{\partial}{\partial \mathbf{z}} T_\varepsilon$ 都是有界的。在以下内容中，我们使用这些条件来交换极限和乘积。

我们可以使用变量离散变化公式以及导数的定义来写出微分方程 $\frac{\partial \log p(\mathbf{z}(t))}{\partial t}$:

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = \lim_{\varepsilon \rightarrow 0^+} \frac{\log p(\mathbf{z}(t)) - \log |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))| - \log p(\mathbf{z}(t))}{\varepsilon} \quad (15)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\log |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|}{\varepsilon} \quad (16)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\frac{\partial}{\partial \varepsilon} \log |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|}{\frac{\partial}{\partial \varepsilon} \varepsilon} \quad (\text{根据洛必达法则}) \quad (17)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\frac{\partial}{\partial \varepsilon} |\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|}{|\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|} \quad \left(\left. \frac{\partial \log(\mathbf{z})}{\partial \mathbf{z}} \right|_{\mathbf{z}=\mathbf{1}} = 1 \right) \quad (18)$$

$$= - \underbrace{\left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \left| \det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right| \right)}_{\text{bounded}} \underbrace{\left(\lim_{\varepsilon \rightarrow 0^+} \frac{1}{|\det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t))|} \right)}_{=1} \quad (19)$$

$$= - \lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \left| \det \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right| \quad (20)$$

行列式的导数可以使用 Jacobi 公式表示，该公式给出

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = - \lim_{\varepsilon \rightarrow 0^+} \text{tr} \left(\text{adj} \left(\frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \quad (21)$$

$$= - \text{tr} \left(\underbrace{\left(\lim_{\varepsilon \rightarrow 0^+} \text{adj} \left(\frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \right)}_{=I} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \right) \quad (22)$$

$$= - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} T_\varepsilon(\mathbf{z}(t)) \right) \quad (23)$$

替代 T_ε 通过它的泰勒级数展开并取极限，我们完成 证据。

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \frac{\partial}{\partial \mathbf{z}} (\mathbf{z} + \varepsilon f(\mathbf{z}(t), t) + \mathcal{O}(\varepsilon^2) + \mathcal{O}(\varepsilon^3) + \dots) \right) \quad (24)$$

$$= - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \frac{\partial}{\partial \varepsilon} \left(I + \frac{\partial}{\partial \mathbf{z}} \varepsilon f(\mathbf{z}(t), t) + \mathcal{O}(\varepsilon^2) + \mathcal{O}(\varepsilon^3) + \dots \right) \right) \quad (25)$$

$$= - \text{tr} \left(\lim_{\varepsilon \rightarrow 0^+} \left(\frac{\partial}{\partial \mathbf{z}} f(\mathbf{z}(t), t) + \mathcal{O}(\varepsilon) + \mathcal{O}(\varepsilon^2) + \dots \right) \right) \quad (26)$$

$$= - \text{tr} \left(\frac{\partial}{\partial \mathbf{z}} f(\mathbf{z}(t), t) \right) \quad (27)$$

□

A.1 Special Cases

Planar CNF. Let $f(\mathbf{z}) = uh(w^z + b)$, then $\frac{\partial f}{\partial \mathbf{z}} = u \frac{\partial h}{\partial \mathbf{z}}^\top$. Since the trace of an outer product is the inner product, we have

$$\frac{\partial \log p(\mathbf{z})}{\partial t} = -\text{tr} \left(u \frac{\partial h}{\partial \mathbf{z}}^\top \right) = -u^\top \frac{\partial h}{\partial \mathbf{z}} \quad (28)$$

This is the parameterization we use in all of our experiments.

Hamiltonian CNF. The continuous analog of NICE (Dinh et al., 2014) is a Hamiltonian flow, which splits the data into two equal partitions and is a volume-preserving transformation, implying that $\frac{\partial \log p(\mathbf{z})}{\partial t} = 0$. We can verify this. Let

$$\begin{bmatrix} \frac{d\mathbf{z}_{1:d}}{dt} \\ \frac{d\mathbf{z}_{d+1:D}}{dt} \end{bmatrix} = \begin{bmatrix} f(\mathbf{z}_{d+1:D}) \\ g(\mathbf{z}_{1:d}) \end{bmatrix} \quad (29)$$

Then because the Jacobian is all zeros on its diagonal, the trace is zero. This is a volume-preserving flow.

A.2 Connection to Fokker-Planck and Liouville PDEs

The Fokker-Planck equation is a well-known partial differential equation (PDE) that describes the probability density function of a stochastic differential equation as it changes with time. We relate the instantaneous change of variables to the special case of Fokker-Planck with zero diffusion, the Liouville equation.

As with the instantaneous change of variables, let $\mathbf{z}(t) \in \mathbb{R}^D$ evolve through time following $\frac{d\mathbf{z}(t)}{dt} = f(\mathbf{z}(t), t)$. Then Liouville equation describes the change in density of $\mathbf{z}$ —a fixed point in space—as a PDE,

$$\frac{\partial p(\mathbf{z}, t)}{\partial t} = - \sum_{i=1}^D \frac{\partial}{\partial \mathbf{z}_i} [f_i(\mathbf{z}, t) p(\mathbf{z}, t)] \quad (30)$$

However, (30) cannot be easily used as it requires the partial derivatives of $\frac{p(\mathbf{z}, t)}{\partial \mathbf{z}}$, which is typically approximated using finite difference. This type of PDE has its own literature on efficient and accurate simulation (Stam, 1999).

Instead of evaluating $p(\cdot, t)$ at a fixed point, if we follow the trajectory of a particle $\mathbf{z}(t)$, we obtain

$$\begin{aligned} \frac{\partial p(\mathbf{z}(t), t)}{\partial t} &= \underbrace{\frac{\partial p(\mathbf{z}(t), t)}{\partial \mathbf{z}(t)} \frac{\partial \mathbf{z}(t)}{\partial t}}_{\text{partial derivative from first argument, } \mathbf{z}(t)} + \underbrace{\frac{\partial p(\mathbf{z}(t), t)}{\partial t}}_{\text{partial derivative from second argument, } t} \\ &= \sum_{i=1}^D \frac{\partial p(\mathbf{z}(t), t)}{\partial \mathbf{z}_i(t)} \frac{\partial \mathbf{z}_i(t)}{\partial t} - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} p(\mathbf{z}(t), t) - \sum_{i=1}^D \cancel{f_i(\mathbf{z}(t), t)} \frac{\partial p(\mathbf{z}(t), t)}{\partial \mathbf{z}_i(t)} \\ &= - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} p(\mathbf{z}(t), t) \end{aligned} \quad (31)$$

We arrive at the instantaneous change of variables by taking the log,

$$\frac{\partial \log p(\mathbf{z}(t), t)}{\partial t} = \frac{1}{p(\mathbf{z}(t), t)} \frac{\partial p(\mathbf{z}(t), t)}{\partial t} = - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} \quad (32)$$

While still a PDE, (32) can be combined with $\mathbf{z}(t)$ to form an ODE of size $D + 1$,

$$\frac{d}{dt} \begin{bmatrix} \mathbf{z}(t) \\ \log p(\mathbf{z}(t), t) \end{bmatrix} = \begin{bmatrix} f(\mathbf{z}(t), t) \\ - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} \end{bmatrix} \quad (33)$$

Compared to the Fokker-Planck and Liouville equations, the instantaneous change of variables is of more practical impact as it can be numerically solved much more easily, requiring an extra state of D for following the trajectory of $\mathbf{z}(t)$. Whereas an approach based on finite difference approximation of the Liouville equation would require a grid size that is exponential in D .

Appendix B A Modern Proof of the Adjoint Method

We present an alternative proof to the adjoint method (Pontryagin et al., 1962) that is short and easy to follow.

A.1 特殊情况

平面 CNF。设 $f(\mathbf{z}) = u h(w^T \mathbf{z} + b)$, 然后 $\frac{\partial f}{\partial \mathbf{z}} = u \frac{\partial h}{\partial \mathbf{z}}^T$ 。由于外积的迹是内积, 我们得到

$$\frac{\partial \log p(\mathbf{z})}{\partial t} = -\text{tr} \left(u \frac{\partial h}{\partial \mathbf{z}}^T \right) = -u^T \frac{\partial h}{\partial \mathbf{z}} \quad (28)$$

这是我们在所有实验中使用的参数化方法。

Hamiltonian CNF。NICE (Dinh 等, 2014) 的连续类似物是哈密顿流, 它将数据分成两个相等的部分, 并且是一种保持体积的变换, 这意味着 $\frac{\partial \log p(\mathbf{z})}{\partial t} = 0$ 。我们可以验证这一点。设

$$\begin{bmatrix} \frac{d\mathbf{z}_{1:d}}{dt} \\ \frac{d\mathbf{z}_{d+1:D}}{dt} \end{bmatrix} = \begin{bmatrix} f(\mathbf{z}_{d+1:D}) \\ g(\mathbf{z}_{1:d}) \end{bmatrix} \quad (29)$$

然后因为雅可比矩阵在对角线上全是零, 所以迹为零。这是一个保持体积的流。

A.2 与福克-普朗克方程和吕维尔偏微分方程的联系

Fokker-Planck 方程是一个著名的偏微分方程 (PDE), 它描述了随机微分方程的概率密度函数随时间的变化。我们将变量的瞬时变化与零扩散的 Fokker-Planck 特殊情况——Liouville 方程联系起来。

与变量的瞬时变化一样, 令 $\mathbf{z}(t) \in \mathbb{R}^D$ 按照 $\frac{d\mathbf{z}(t)}{dt} = f(\mathbf{z}(t), t)$ 随时间演化。然后李乌维尔方程将 $\mathbf{z}$ - a fixed point in space 的密度变化描述为一个偏微分方程 (PDE),

$$\frac{\partial p(\mathbf{z}, t)}{\partial t} = - \sum_{i=1}^D \frac{\partial}{\partial \mathbf{z}_i} [f_i(\mathbf{z}, t) p(\mathbf{z}, t)] \quad (30)$$

然而, (30) 不容易使用, 因为它需要 $\frac{\partial p(\mathbf{z}, t)}{\partial \mathbf{z}}$ 的偏导数, 这通常使用有限差分来近似。这类偏微分方程在高效且精确的模拟方面有其自己的文献 (Stam, 1999)。

与其在固定点评估 $p(\cdot, t)$, 如果我们跟随粒子 $\mathbf{z}(t)$ 的轨迹, 我们得到

$$\begin{aligned} \frac{\partial p(\mathbf{z}(t), t)}{\partial t} &= \underbrace{\frac{\partial p(\mathbf{z}(t), t)}{\partial \mathbf{z}(t)} \frac{\partial \mathbf{z}(t)}{\partial t}}_{\text{partial derivative from first argument, } \mathbf{z}(t)} + \underbrace{\frac{\partial p(\mathbf{z}(t), t)}{\partial t}}_{\text{partial derivative from second argument, } t} \\ &= \sum_{i=1}^D \frac{\partial p(\mathbf{z}(t), t)}{\partial \mathbf{z}_i(t)} \frac{\partial \mathbf{z}_i(t)}{\partial t} - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} p(\mathbf{z}(t), t) - \sum_{i=1}^D f_i(\mathbf{z}(t), t) \frac{\partial p(\mathbf{z}(t), t)}{\partial \mathbf{z}_i(t)} \\ &= - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} p(\mathbf{z}(t), t) \end{aligned} \quad (31)$$

我们通过取对数得到变量的瞬时变化,

$$\frac{\partial \log p(\mathbf{z}(t), t)}{\partial t} = \frac{1}{p(\mathbf{z}(t), t)} \frac{\partial p(\mathbf{z}(t), t)}{\partial t} = - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} \quad (32)$$

虽然仍然是偏微分方程, (32) 可以与 $\mathbf{z}(t)$ 结合形成一个大小为 $D + 1$ 的常微分方程 1,

$$\frac{d}{dt} \begin{bmatrix} \mathbf{z}(t) \\ \log p(\mathbf{z}(t), t) \end{bmatrix} = \begin{bmatrix} f(\mathbf{z}(t), t) \\ - \sum_{i=1}^D \frac{\partial f_i(\mathbf{z}(t), t)}{\partial \mathbf{z}_i} \end{bmatrix} \quad (33)$$

与 Fokker-Planck 方程和 Liouville 方程相比, 变量的瞬时变化具有更实际的影响, 因为它可以更容易地进行数值求解, 只需额外的 D 状态即可跟踪 $\mathbf{z}(t)$ 的轨迹。而基于 Liouville 方程的有限差分近似方法则需要一个在 D 上呈指数增长的网格尺寸。

附录B 伴随方法的现代证明

我们提出了一种对伴随方法 (Pontryagin 等, 1962) 的替代证明, 该证明简短且易于理解。

B.1 Continuous Backpropagation

Let $\mathbf{z}(t)$ follow the differential equation $\frac{d\mathbf{z}(t)}{dt} = f(\mathbf{z}(t), t, \theta)$, where θ are the parameters. We will prove that if we define an adjoint state

$$\mathbf{a}(t) = \frac{dL}{d\mathbf{z}(t)} \quad (34)$$

then it follows the differential equation

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} \quad (35)$$

For ease of notation, we denote vectors as row vectors, whereas the main text uses column vectors.

The adjoint state is the gradient with respect to the hidden state at a specified time t . In standard neural networks, the gradient of a hidden layer $\mathbf{h}_t$ depends on the gradient from the next layer $\mathbf{h}_{t+1}$ by chain rule

$$\frac{dL}{d\mathbf{h}_t} = \frac{dL}{d\mathbf{h}_{t+1}} \frac{d\mathbf{h}_{t+1}}{d\mathbf{h}_t}. \quad (36)$$

With a continuous hidden state, we can write the transformation after an ε change in time as

$$\mathbf{z}(t + \varepsilon) = \int_t^{t+\varepsilon} f(\mathbf{z}(t), t, \theta) dt + \mathbf{z}(t) = T_\varepsilon(\mathbf{z}(t), t) \quad (37)$$

and chain rule can also be applied

$$\frac{dL}{d\mathbf{z}(t)} = \frac{dL}{d\mathbf{z}(t + \varepsilon)} \frac{d\mathbf{z}(t + \varepsilon)}{d\mathbf{z}(t)} \quad \text{or} \quad \mathbf{a}(t) = \mathbf{a}(t + \varepsilon) \frac{\partial T_\varepsilon(\mathbf{z}(t), t)}{\partial \mathbf{z}(t)} \quad (38)$$

The proof of (35) follows from the definition of derivative:

$$\frac{d\mathbf{a}(t)}{dt} = \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t)}{\varepsilon} \quad (39)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t + \varepsilon) \frac{\partial}{\partial \mathbf{z}(t)} T_\varepsilon(\mathbf{z}(t))}{\varepsilon} \quad (\text{by Eq 38}) \quad (40)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t + \varepsilon) \frac{\partial}{\partial \mathbf{z}(t)} (\mathbf{z}(t) + \varepsilon f(\mathbf{z}(t), t, \theta) + \mathcal{O}(\varepsilon^2))}{\varepsilon} \quad (\text{Taylor series around } \mathbf{z}(t)) \quad (41)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t + \varepsilon) \left(I + \varepsilon \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon^2) \right)}{\varepsilon} \quad (42)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{-\varepsilon \mathbf{a}(t + \varepsilon) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon^2)}{\varepsilon} \quad (43)$$

$$= \lim_{\varepsilon \rightarrow 0^+} -\mathbf{a}(t + \varepsilon) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon) \quad (44)$$

$$= -\mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} \quad (45)$$

We pointed out the similarity between adjoint method and backpropagation (eq. 38). Similarly to backpropagation, ODE for the adjoint state needs to be solved *backwards* in time. We specify the constraint on the last time point, which is simply the gradient of the loss wrt the last time point, and can obtain the gradients with respect to the hidden state at any time, including the initial value.

$$\underbrace{\mathbf{a}(t_N) = \frac{dL}{d\mathbf{z}(t_N)}}_{\text{initial condition of adjoint diffeq.}} \quad \underbrace{\mathbf{a}(t_0) = \mathbf{a}(t_N) + \int_{t_N}^{t_0} \frac{d\mathbf{a}(t)}{dt} dt = \mathbf{a}(t_N) - \int_{t_N}^{t_0} \mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} dt}_{\text{gradient wrt. initial value}} \quad (46)$$

Here we assumed that loss function L depends only on the last time point t_N . If function L depends also on intermediate time points $t_1, t_2, \dots, t_{N-1}$, etc., we can repeat the adjoint step for each of the intervals $[t_{N-1}, t_N]$, $[t_{N-2}, t_{N-1}]$ in the backward order and sum up the obtained gradients.

B.2 Gradients wrt. θ and t

We can generalize (35) to obtain gradients with respect to θ —a constant wrt. t —and the initial and end times, t_0 and t_N . We view θ and t as states with constant differential equations and write

$$\frac{\partial \theta(t)}{\partial t} = \mathbf{0} \quad \frac{dt(t)}{dt} = 1 \quad (47)$$

B.1 连续反向传播

令 $\mathbf{z}(t)$ 遵循微分方程 $\frac{d\mathbf{z}(t)}{dt} = f(\mathbf{z}(t), t, \theta)$, 其中 θ 是参数。我们将证明, 如果我们定义一个伴随状态

$$\mathbf{a}(t) = \frac{dL}{d\mathbf{z}(t)} \quad (34)$$

那么它符合微分方程

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} \quad (35)$$

为了便于表示, 我们将向量表示为行向量, 而正文中使用列向量。

伴随状态是在指定时间 t 对隐藏状态的梯度。在标准神经网络中, 隐藏层 $\mathbf{h}_t$ 的梯度依赖于下一层 $\mathbf{h}_{t+1}$ 的梯度, 通过链式法则计算

$$\frac{dL}{d\mathbf{h}_t} = \frac{dL}{d\mathbf{h}_{t+1}} \frac{d\mathbf{h}_{t+1}}{d\mathbf{h}_t}. \quad (36)$$

对于连续的隐状态, 我们可以将经过时间上的 ε 变化后的变换写为

$$\mathbf{z}(t + \varepsilon) = \int_t^{t+\varepsilon} f(\mathbf{z}(t), t, \theta) dt + \mathbf{z}(t) = T_\varepsilon(\mathbf{z}(t), t) \quad (37)$$

并且链式法则也可以应用

$$\frac{dL}{d\mathbf{z}(t)} = \frac{dL}{d\mathbf{z}(t + \varepsilon)} \frac{d\mathbf{z}(t + \varepsilon)}{d\mathbf{z}(t)} \quad \text{or} \quad \mathbf{a}(t) = \mathbf{a}(t + \varepsilon) \frac{\partial T_\varepsilon(\mathbf{z}(t), t)}{\partial \mathbf{z}(t)} \quad (38)$$

(35)的证明来源于导数的定义:

$$\frac{d\mathbf{a}(t)}{dt} = \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t)}{\varepsilon} \quad (39)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t + \varepsilon) \frac{\partial}{\partial \mathbf{z}(t)} T_\varepsilon(\mathbf{z}(t))}{\varepsilon} \quad (\text{by Eq 38}) \quad (40)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t + \varepsilon) \frac{\partial}{\partial \mathbf{z}(t)} (\mathbf{z}(t) + \varepsilon f(\mathbf{z}(t), t, \theta) + \mathcal{O}(\varepsilon^2))}{\varepsilon} \quad (\text{Taylor series around } \mathbf{z}(t)) \quad (41)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{\mathbf{a}(t + \varepsilon) - \mathbf{a}(t + \varepsilon) \left(I + \varepsilon \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon^2) \right)}{\varepsilon} \quad (42)$$

$$= \lim_{\varepsilon \rightarrow 0^+} \frac{-\varepsilon \mathbf{a}(t + \varepsilon) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon^2)}{\varepsilon} \quad (43)$$

$$= \lim_{\varepsilon \rightarrow 0^+} -\mathbf{a}(t + \varepsilon) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon) \quad (44)$$

$$= -\mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} \quad (45)$$

我们指出了伴随法与反向传播的相似性 (公式38)。与反向传播类似, 伴随状态的ODE需要随时间求解。我们在最后一个时间点指定约束, 这只是损失函数对最后一个时间点的梯度, 并且可以获得相对于任意时间点的隐藏状态的梯度, 包括初始值。

$$\underbrace{\mathbf{a}(t_N) = \frac{dL}{d\mathbf{z}(t_N)}}_{\text{伴随微分方程的初始条件.}} \quad \underbrace{\mathbf{a}(t_0) = \mathbf{a}(t_N) + \int_{t_N}^{t_0} \frac{d\mathbf{a}(t)}{dt} dt = \mathbf{a}(t_N) - \int_{t_N}^{t_0} \mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)}}_{\text{关于初始值的梯度}} \quad (46)$$

这里我们假设损失函数 L 仅依赖于最后的时间点 t_N 。如果函数 L 也依赖于中间时间点 $t_1, t_2, \dots, t_{N-1}$ 等, 我们可以对每个区间 $[t_{N-1}, t_N]$ 、 $[t_{N-2}, t_{N-1}]$ 按逆序重复伴随步骤, 并将得到的梯度相加。

B.2 关于 θ 和 t 的梯度

我们可以将 (35) 泛化, 以获得关于 θ (相对于 t 是常数) 以及初始和结束时间 t_0 和 t_N 的梯度。我们将 θ 和 t 视为具有常微分方程的状态, 并写出

$$\frac{\partial \theta(t)}{\partial t} = \mathbf{0} \quad \frac{dt(t)}{dt} = 1 \quad (47)$$

We can then combine these with z to form an augmented state¹ with corresponding differential equation and adjoint state,

$$\frac{d}{dt} \begin{bmatrix} \mathbf{z} \\ \theta \\ t \end{bmatrix} (t) = f_{aug}([\mathbf{z}, \theta, t]) := \begin{bmatrix} f([\mathbf{z}, \theta, t]) \\ \mathbf{0} \\ 1 \end{bmatrix}, \mathbf{a}_{aug} := \begin{bmatrix} \mathbf{a} \\ \mathbf{a}_\theta \\ \mathbf{a}_t \end{bmatrix}, \mathbf{a}_\theta(t) := \frac{dL}{d\theta(t)}, \mathbf{a}_t(t) := \frac{dL}{dt(t)} \quad (48)$$

Note this formulates the augmented ODE as an autonomous (time-invariant) ODE, but the derivations in the previous section still hold as this is a special case of a time-variant ODE. The Jacobian of f has the form

$$\frac{\partial f_{aug}}{\partial [\mathbf{z}, \theta, t]} = \begin{bmatrix} \frac{\partial f}{\partial \mathbf{z}} & \frac{\partial f}{\partial \theta} & \frac{\partial f}{\partial t} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} (t) \quad (49)$$

where each $\mathbf{0}$ is a matrix of zeros with the appropriate dimensions. We plug this into (35) to obtain

$$\frac{d\mathbf{a}_{aug}(t)}{dt} = -[\mathbf{a}(t) \quad \mathbf{a}_\theta(t) \quad \mathbf{a}_t(t)] \frac{\partial f_{aug}}{\partial [\mathbf{z}, \theta, t]}(t) = -[\mathbf{a} \frac{\partial f}{\partial \mathbf{z}} \quad \mathbf{a} \frac{\partial f}{\partial \theta} \quad \mathbf{a} \frac{\partial f}{\partial t}](t) \quad (50)$$

The first element is the adjoint differential equation (35), as expected. The second element can be used to obtain the total gradient with respect to the parameters, by integrating over the full interval and setting $\mathbf{a}_\theta(t_N) = \mathbf{0}$.

$$\frac{dL}{d\theta} = \mathbf{a}_\theta(t_0) = - \int_{t_N}^{t_0} \mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt \quad (51)$$

Finally, we also get gradients with respect to t_0 and t_N , the start and end of the integration interval.

$$\frac{dL}{dt_N} = \mathbf{a}(t_N) f(\mathbf{z}(t_N), t_N, \theta) \quad \frac{dL}{dt_0} = \mathbf{a}_t(t_0) = \mathbf{a}_t(t_N) - \int_{t_N}^{t_0} \mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial t} dt \quad (52)$$

Between (35), (46), (51), and (52) we have gradients for all possible inputs to an initial value problem solver.

Appendix C Full Adjoint sensitivities algorithm

This more detailed version of Algorithm 1 includes gradients with respect to the start and end times of integration.

Algorithm 2 Complete reverse-mode derivative of an ODE initial value problem

Input: dynamics parameters θ , start time t_0 , stop time t_1 , final state $\mathbf{z}(t_1)$, loss gradient $\partial L / \partial \mathbf{z}(t_1)$

$\frac{\partial L}{\partial t_1} = \frac{\partial L}{\partial \mathbf{z}(t_1)}^\top f(\mathbf{z}(t_1), t_1, \theta)$ ▷ Compute gradient w.r.t. t_1

$s_0 = [\mathbf{z}(t_1), \frac{\partial L}{\partial \mathbf{z}(t_1)}, \mathbf{0}_{|\theta|}, -\frac{\partial L}{\partial t_1}]$ ▷ Define initial augmented state

def aug_dynamics($[\mathbf{z}(t), \mathbf{a}(t), \cdot, \cdot], t, \theta$): ▷ Define dynamics on augmented state

return $[f(\mathbf{z}(t), t, \theta), -\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{z}}, -\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}, -\mathbf{a}(t)^\top \frac{\partial f}{\partial t}]$ ▷ Compute vector-Jacobian products

$[\mathbf{z}(t_0), \frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}] = \text{ODESolve}(s_0, \text{aug_dynamics}, t_1, t_0, \theta)$ ▷ Solve reverse-time ODE

return $\frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}, \frac{\partial L}{\partial t_1}$ ▷ Return all gradients

¹Note that we've overloaded t to be both a part of the state and the (dummy) independent variable. The distinction is clear given context, so we keep t as the independent variable for consistency with the rest of the text.

然后我们可以将这些与 z 结合，形成一个扩展状态¹，并对应相应的微分方程和伴随状态，

$$\frac{d}{dt} \begin{bmatrix} \mathbf{z} \\ \theta \\ t \end{bmatrix} (t) = f_{aug}([\mathbf{z}, \theta, t]) := \begin{bmatrix} f([\mathbf{z}, \theta, t]) \\ \mathbf{0} \\ 1 \end{bmatrix}, \mathbf{a}_{aug} := \begin{bmatrix} \mathbf{a} \\ \mathbf{a}_\theta \\ \mathbf{a}_t \end{bmatrix}, \mathbf{a}_\theta(t) := \frac{dL}{d\theta(t)}, \mathbf{a}_t(t) := \frac{dL}{dt(t)} \quad (48)$$

请注意，这将增广的常微分方程公式化为自治（时间不变）常微分方程，但上一节中的推导仍然成立，因为这是时间可变常微分方程的一个特例。 f 的雅可比矩阵形式为

$$\frac{\partial f_{aug}}{\partial [\mathbf{z}, \theta, t]} = \begin{bmatrix} \frac{\partial f}{\partial \mathbf{z}} & \frac{\partial f}{\partial \theta} & \frac{\partial f}{\partial t} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} (t) \quad (49)$$

其中每个 $\mathbf{0}$ 是具有适当维度的零矩阵。我们将其代入 (35) 以得到

$$\frac{d\mathbf{a}_{aug}(t)}{dt} = -[\mathbf{a}(t) \quad \mathbf{a}_\theta(t) \quad \mathbf{a}_t(t)] \frac{\partial f_{aug}}{\partial [\mathbf{z}, \theta, t]}(t) = -[\mathbf{a} \frac{\partial f}{\partial \mathbf{z}} \quad \mathbf{a} \frac{\partial f}{\partial \theta} \quad \mathbf{a} \frac{\partial f}{\partial t}](t) \quad (50)$$

第一个元素是伴随微分方程 (35)，这是预期中的。第二个元素可以用于通过对整个区间积分并设置 $\mathbf{a}_\theta(t_N) = \mathbf{0}$ 来获得关于参数的总梯度。

$$\frac{dL}{d\theta} = \mathbf{a}_\theta(t_0) = - \int_{t_N}^{t_0} \mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt \quad (51)$$

最后，我们还得到相对于 t_0 和 t_N 的梯度，即积分区间的起点和终点。

$$\frac{dL}{dt_N} = \mathbf{a}(t_N) f(\mathbf{z}(t_N), t_N, \theta) \quad \frac{dL}{dt_0} = \mathbf{a}_t(t_0) = \mathbf{a}_t(t_N) - \int_{t_N}^{t_0} \mathbf{a}(t) \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial t} dt \quad (52)$$

在 (35)、(46)、(51) 和 (52) 之间，我们拥有一个初值问题求解器对所有可能输入的梯度。

附录C 全伴随灵敏度算法

算法1的这个更详细版本包括关于积分起止时间的梯度。

Algorithm 2 Complete reverse-mode derivative of an ODE initial value problem

Input: dynamics parameters θ , start time t_0 , stop time t_1 , final state $\mathbf{z}(t_1)$, loss gradient $\partial L / \partial \mathbf{z}(t_1)$

$\frac{\partial L}{\partial t_1} = \frac{\partial L}{\partial \mathbf{z}(t_1)}^\top f(\mathbf{z}(t_1), t_1, \theta)$ ▷ Compute gradient w.r.t. t_1

$s_0 = [\mathbf{z}(t_1), \frac{\partial L}{\partial \mathbf{z}(t_1)}, \mathbf{0}_{|\theta|}, -\frac{\partial L}{\partial t_1}]$ ▷ Define initial augmented state

def aug_dynamics($[\mathbf{z}(t), \mathbf{a}(t), \cdot, \cdot], t, \theta$): ▷ Define dynamics on augmented state

return $[f(\mathbf{z}(t), t, \theta), -\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{z}}, -\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}, -\mathbf{a}(t)^\top \frac{\partial f}{\partial t}]$ ▷ Compute vector-Jacobian products

$[\mathbf{z}(t_0), \frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}] = \text{ODESolve}(s_0, \text{aug_dynamics}, t_1, t_0, \theta)$ ▷ Solve reverse-time ODE

return $\frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}, \frac{\partial L}{\partial t_1}$ ▷ Return all gradients

¹Note that we've overloaded t to be both a part of the state and the (dummy) independent variable. The distinction is clear given context, so we keep t as the independent variable for consistency with the rest of the text.

Appendix D Autograd Implementation

```
import scipy.integrate

import autograd.numpy as np
from autograd.extend import primitive, defvjp_argnums
from autograd import make_vjp
from autograd.misc import flatten
from autograd.builtins import tuple

odeint = primitive(scipy.integrate.odeint)

def grad_odeint_all(yt, func, y0, t, func_args, **kwargs):
    # Extended from "Scalable Inference of Ordinary Differential
    # Equation Models of Biochemical Processes", Sec. 2.4.2
    # Fabian Froehlich, Carolin Loos, Jan Hasenauer, 2017
    # https://arxiv.org/pdf/1711.08079.pdf

    T, D = np.shape(yt)
    flat_args, unflatten = flatten(func_args)

    def flat_func(y, t, flat_args):
        return func(y, t, *unflatten(flat_args))

    def unpack(x):
        # y, vjp_y, vjp_t, vjp_args
        return x[0:D], x[D:2 * D], x[2 * D], x[2 * D + 1:]

    def augmented_dynamics(augmented_state, t, flat_args):
        # Original system augmented with vjp_y, vjp_t and vjp_args.
        y, vjp_y, _, _ = unpack(augmented_state)
        vjp_all, dy_dt = make_vjp(flat_func, argnum=(0, 1, 2))(y, t, flat_args)
        vjp_y, vjp_t, vjp_args = vjp_all(-vjp_y)
        return np.hstack((dy_dt, vjp_y, vjp_t, vjp_args))

    def vjp_all(g, **kwargs):
        vjp_y = g[-1, :]
        vjp_t0 = 0
        time_vjp_list = []
        vjp_args = np.zeros(np.size(flat_args))

        for i in range(T - 1, 0, -1):
            # Compute effect of moving current time.
            vjp_cur_t = np.dot(func(yt[i, :], t[i], *func_args), g[i, :])
            time_vjp_list.append(vjp_cur_t)
            vjp_t0 = vjp_t0 - vjp_cur_t

            # Run augmented system backwards to the previous observation.
            aug_y0 = np.hstack((yt[i, :], vjp_y, vjp_t0, vjp_args))
            aug_ans = odeint(augmented_dynamics, aug_y0,
                             np.array([t[i], t[i - 1]]), tuple((flat_args,)), **kwargs)
            _, vjp_y, vjp_t0, vjp_args = unpack(aug_ans[1])

            # Add gradient from current output.
            vjp_y = vjp_y + g[i - 1, :]

        time_vjp_list.append(vjp_t0)
        vjp_times = np.hstack(time_vjp_list)[::-1]

        return None, vjp_y, vjp_times, unflatten(vjp_args)
    return vjp_all
```

附录 D 自动微分实现

导入 `scipy.integrate`

导入自动化。numpy 作为 np
f 从 `autograd.ext` 导入 `primitive`, `def vjp_a` `r g n u m s`
从 `autograd` 导入 `make_vjp`
来自 `autograd.misc` 导入 `flatten`
来自 `autograd` 的内置导入元组

`odeint` = 原始(`scipy.integrate.odeint`)

`def grad_odeint_all(yt, func, y0, t, func_args, **kwargs):` # 扩展自 “可
扩展生化过程常微分方程模型推理”, 第2.4.2节 # Fabian Froehlich, Carolin Loos, Ja
n Hasenauer, 2017 # <https://arxiv.org/pdf/1711.08079.pdf> T, D = np.shape(yt) flat_args
, unflatten = flatten (func_args)

`def flat_func(y, t, flat_args):` 返回 `func(y,`
`t, *unflatten(flat_args))`

`def 解包(x):` # y, vjp_y, vjp_t, vjp_args 返回 `x[0:D], x[D:2*D]`
`], x[2*D], x[2*D+1:]`

`def 增强_动力学(增强_状态, t, 平坦_参数):` # 用 vjp_y、vjp_t 和 vjp_参数增强原始系统。y, vjp_y, _ =
解包(增强_状态) vjp_all, dy_dt = 创建_vjp(平坦_函数, 参数编号=(0, 1, 2))(y, t, 平坦_参数) vjp_y, vjp_t,
vjp_参数 = vjp_all(-vjp_y) 返回 `np.hstack((dy_dt, vjp_y, vjp_t, vjp_参数))`

`def vjp_all(g, **关键字参数):`

`vjp_y = g[-1, :] vjp_t0 = 0 time_vjp_list = [] vjp_args = np.zeros(np.size(flat_arg`
`s)) for i in range(T-1, 0, -1):` # 计算移动当前时间的效果 `vjp_cur_t = np.dot(func(yt[i, :], t[i]`
`, *func_args), g[i, :]) time_vjp_list.append(vjp_cur_t) vjp_t0 = vjp_t0 - vjp_cur_t` # 运行增
强系统向后到先前的观测 `aug_y0 = np.hstack((yt[i, :], vjp_y, vjp_t0, vjp_args)) aug_ans = odeint(augm`
`ented_dynamics, aug_y0, np.array([t[i], t[i-1]]), tuple((flat_args,)), **kwargs)` _, vjp_y, vjp
_t0, vjp_args = `unpack(aug_ans[1])` # 从当前输出添加梯度 `vjp_y`

```

def grad_argnums_wrapper(all_vjp_builder):
    # A generic autograd helper function. Takes a function that
    # builds vjps for all arguments, and wraps it to return only required vjps.
    def build_selected_vjps(argnums, ans, combined_args, kwargs):
        vjp_func = all_vjp_builder(ans, *combined_args, **kwargs)
        def chosen_vjps(g):
            # Return whichever vjps were asked for.
            all_vjps = vjp_func(g)
            return [all_vjps[argnum] for argnum in argnums]
        return chosen_vjps
    return build_selected_vjps

defvjp_argnums(odeint, grad_argnums_wrapper(grad_odeint_all))

```

Appendix E Algorithm for training the latent ODE model

To obtain the latent representation $\mathbf{z}_{t_0}$, we traverse the sequence using RNN and obtain parameters of distribution $q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \theta_{enc})$. The algorithm follows a standard VAE algorithm with an RNN variational posterior and an ODESolve model:

1. Run an RNN encoder through the time series and infer the parameters for a posterior over $\mathbf{z}_{t_0}$:

$$q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \phi) = \mathcal{N}(\mathbf{z}_{t_0}|\mu_{\mathbf{z}_{t_0}}, \sigma_{\mathbf{z}_{t_0}}), \quad (53)$$

where $\mu_{\mathbf{z}_{t_0}}, \sigma_{\mathbf{z}_{t_0}}$ comes from hidden state of $\text{RNN}(\{\mathbf{x}_{t_i}, t_i\}_i, \phi)$

2. Sample $\mathbf{z}_{t_0} \sim q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i)$
3. Obtain $\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \dots, \mathbf{z}_{t_M}$ by solving ODE $\text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \dots, t_M)$, where f is the function defining the gradient $d\mathbf{z}/dt$ as a function of $\mathbf{z}$
4. Maximize $\text{ELBO} = \sum_{i=1}^M \log p(\mathbf{x}_{t_i}|\mathbf{z}_{t_i}, \theta_x) + \log p(\mathbf{z}_{t_0}) - \log q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \phi)$, where $p(\mathbf{z}_{t_0}) = \mathcal{N}(0, 1)$

Appendix F Extra Figures

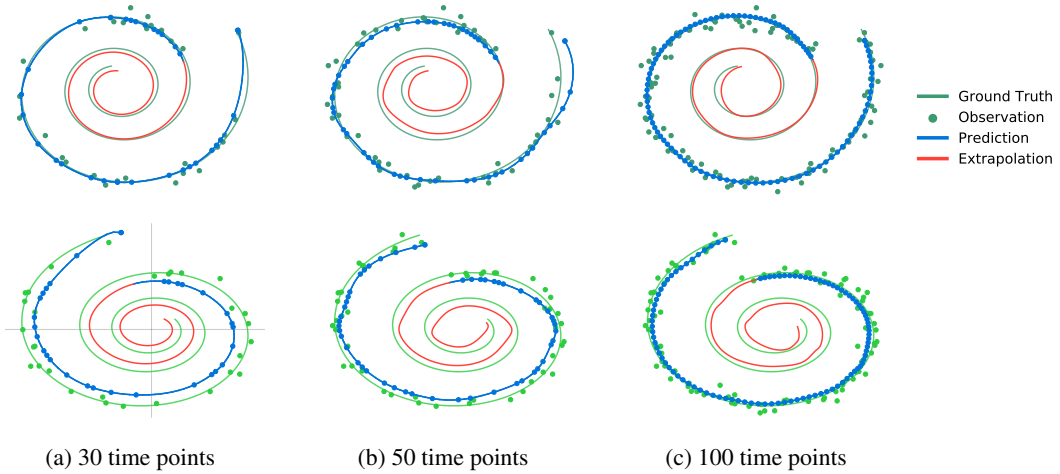


Figure 10: Spiral reconstructions using a latent ODE with a variable number of noisy observations.

```
def grad_argnums_wrapper(all_vjp_builder): # 一个通用的自动梯度辅助函数。接收一个
函数，该函数# 为所有参数构建 VJPs，并将其包装以仅返回所需的 VJPs。
def build_selected_vjps(argnums, ans, combined_args, kwargs):
    vjp_func = all_vjp_builder(ans, *combined_args, **kwargs)
    def chosen_vjps(g): # 返回所请求的 VJPs。
        all_vjps = vjp_func(g)
        return [all_vjps[argnum] for argnum in argnums]
    return chosen_vjps
return build_selected_vjps
```

```
def vjp_argnums(odeint, grad_argnums_wrapper(grad_odeint_all))
```

附录E 潜在ODE模型训练算法

为了获得潜在表示 $\mathbf{z}_{t_0}$ ，我们使用 RNN 遍历序列并获得分布 $q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \theta_{enc})$ 的参数。该算法遵循标准的 VAE 算法，具有 RNN 变分后验和 ODEsolve 模型：

1. 运行一个循环神经网络序列的编码器并推断海报的参数

ior 超过 $\mathbf{z}_{t_0}$:

$$q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \phi) = \mathcal{N}(\mathbf{z}_{t_0}|\mu_{\mathbf{z}_{t_0}}, \sigma_{\mathbf{z}_0}), \quad (53)$$

$\mu_{\mathbf{z}_0}, \sigma_{\mathbf{z}_0}$ 来自 $\text{RNN}(\{\mathbf{x}_{t_i}, t_i\}_i, \phi)$ 的隐藏状态

2. 样本 $\mathbf{z}_{t_0} \sim q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i)$
3. 通过求解 ODE $\text{ODEsolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \dots, t_M)$ 得到 $\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \dots, \mathbf{z}_{t_M}$ ，其中 f 是定义梯度 $d\mathbf{z}/dt$ 关于 $\mathbf{z}$ 的函数
4. 最大化 $\text{ELBO} = \sum_{i=1}^M \log p(\mathbf{x}_{t_i}|\mathbf{z}_{t_i}, \theta_x) + \log p(\mathbf{z}_{t_0}) - \log q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \phi)$ ，其中 $p(\mathbf{z}_{t_0}) = \mathcal{N}(0, 1)$

附录 F 额外图表

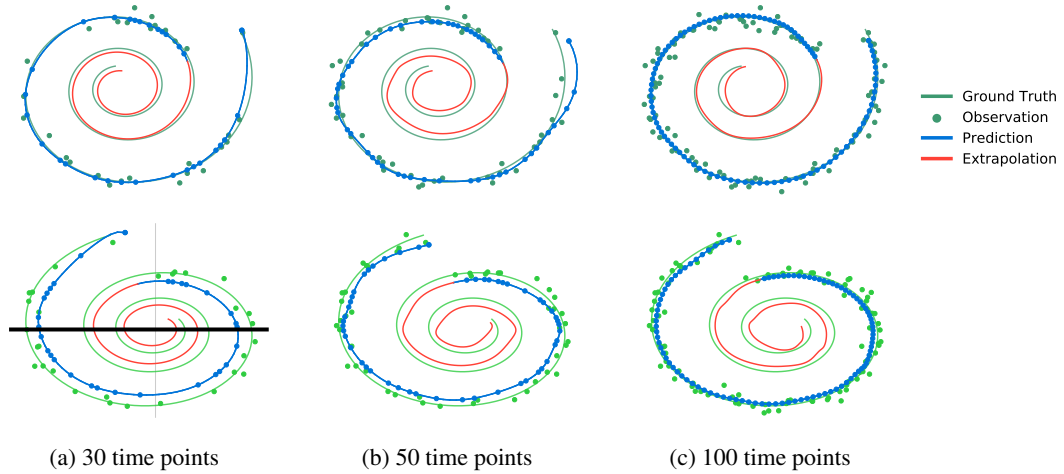


图10：使用潜在ODE进行螺旋重建，具有可变数量的噪声观测。